

Corso di Architettura degli Elaboratori a.a. 2025/2026

L'Instruction Set Architecture (ISA) RISC-V

RISC-V Instruction set

- Proposto e sviluppato all'Università di Berkeley dal 2010 da Andrew Waterman, Yunsup Lee, Krste Asanovic
- Obiettivi
 - Costruire un hardware semplice
 - Compilatori efficienti
 - Massimizzare le prestazioni
 - Minimizzare costi
 - Minimizzare il consumo energetico
- Standard aperto

RISC-V ISA

- ISA semplice che può essere esteso
- License-free, royalty-free RISC ISA
- Standard mantenuto dalla RISC-V Foundation (fondata nel 2015)
- Specifiche adatte a dispositivi eterogenei (dai microcontrollori ai supercomputer)
- RISC-V non è
 - Una azienda
 - Una CPU

RISC-V ISA

- ISA semplice che può essere esteso
- Esempio
 - **RV32I: base integer 32 bit instruction set**
 - **RV64I: base integer 64 bit instruction set**
 - RV64M: aggiunta delle istruzioni per **moltiplicazioni e divisioni** intere
 - RV64F: aggiunta delle istruzioni per la gestione dei **numeri in virgola mobile**
 - RV64E / RV32E (16 registri)

Extension	Description
I	Integer
M	Integer Multiplication and Division
A	Atomics
F	Single-Precision Floating Point
D	Double-Precision Floating Point
G	General Purpose = IMAFD
C	16-bit Compressed Instructions
Non-Standard User-Level Extensions	
Xext	Non-standard extension "ext"

Common RISC-V Standard Extensions

*Not a complete list

RISC-V Innovation Roadmap

Industry Adoption

Proliferation of RISC-V CPUs across performance and application spectrum
RISC-V dominant in universities
Strategic and growing adoption in HPC, automotive, transportation, cloud, industrial, communications, IoT, enterprise, consumer, and other applications

Test Chips Software tests Linux port	Proof of Concept SoCs Minion processors for power management, communications Bare metal software	IoT SoCs Microcontrollers RTOS, Firmware Development tools Technical Steering Committee, HPC SIG, GlobalPlatform partnership	AI SoCs, Application processors, Linux Drivers, AI Compilers Dev Board program Development Partners RISC-V Labs, Security response process, AI SIG, Graphics SIG, Android SIG, Communications SIG
--	--	--	--

2010 – 2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	➔
ISA Definition RISC-V Foundation	RV32	RV32I and RV64I Base instructions: Integer, floating point, multiply and divide, atomic, and compact instructions Priv modes, Interrupts, exceptions, memory model, protection, and virtual memory	Arch compatibility framework, Processor trace		Zfinx ZiHintPause BitManip Vector RISC-V Profiles & Platforms Crypto Scalar Virtual Memory Hypervisor & Advanced interrupt architecture Cache mgt ops Code size reduction* Trusted Execution Environment* P (Packed SIMD)*		RV32E and RV64E 64 bit and 128 bit addresses* Vector Atomic and quad-widening* Quad floating point in integer registers* Crypto Vector* Trusted Execution phase 2* Jit pointer masking & I/D synch* BitManip phase 2* Cache management phase 2* ... and more			

Technical Deliverables



* On track, subject to change

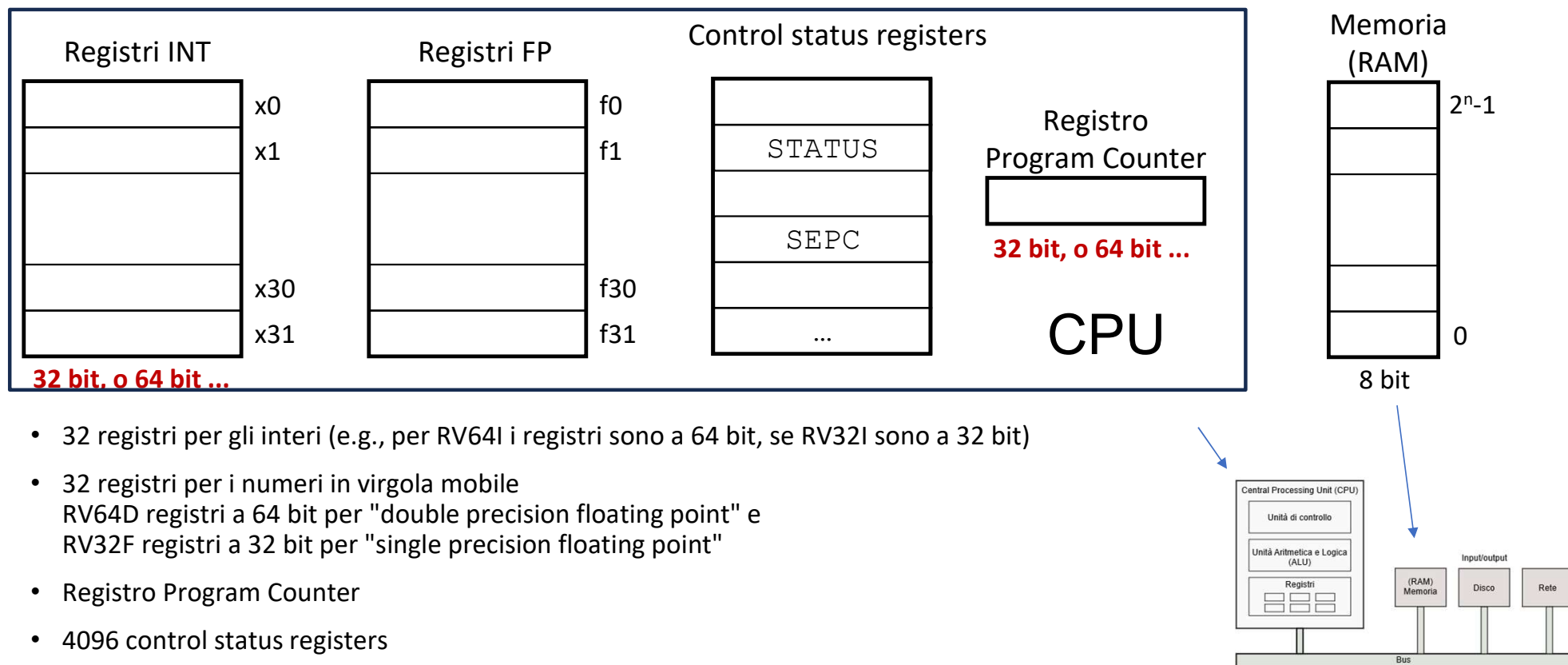
RISC-V Instruction set

- RISC
 - Reduced Instruction Set Computer
- Principi di progettazione
 1. La semplicità favorisce la regolarità
 2. Minori sono le dimensioni, maggiore è la velocità
 3. Un buon progetto richiede buoni compromessi

RISC-V Registri e memoria

Parola (word): 32 bit

Parola doppia (doubleword): 64 bit



- 32 registri per gli interi (e.g., per RV64I i registri sono a 64 bit, se RV32I sono a 32 bit)
- 32 registri per i numeri in virgola mobile
RV64D registri a 64 bit per "double precision floating point" e
RV32F registri a 32 bit per "single precision floating point"
- Registro Program Counter
- 4096 control status registers
- **Memoria centrale (ad esempio, 2^{64} indirizzi nella RV64I, 2^{32} indirizzi nella RV32I)**

RISC-V Instruction Set

RISC-V Registri e memoria

Parola (word): 32 bit

Parola doppia (doubleword): 64 bit

- Registri per gli interi
 - Quantità: 32, indicati con x0 .. X31
 - Dimensione: 64 bit se RV64I, 32 bit se RV32I
 -

x0	zero
x1	Return address (ra)
x2	Stack pointer (sp)
x3	Global pointer (gp)
x4	Thread pointer (tp)
x8	Frame pointer (fp)
x10-x17	Registri usati per il passaggio di parametri nelle procedure e valori di ritorno
x5-x7 , x28-x31	Registri temporanei, non salvati in caso di chiamata
x8-x9, x18-x27	Registri da salvare: il contenuto va preservato se utilizzati dalla procedura chiamata

RISC-V Instruction set

- Tipologie di istruzione
 - Aritmetiche
 - Logiche
 - Accesso alla memoria
 - Condizionali

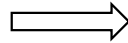
Istruzioni aritmetiche

- Notazione rigida
 - Tutte le istruzioni aritmetiche hanno esattamente 3 operandi
 - L'ordine degli operandi è fisso

- Addizione

`a = b + c`

Linguaggio C



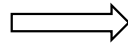
`add a, b, c`

RISC-V assembly

- Sottrazione

`a = b - c`

Linguaggio C

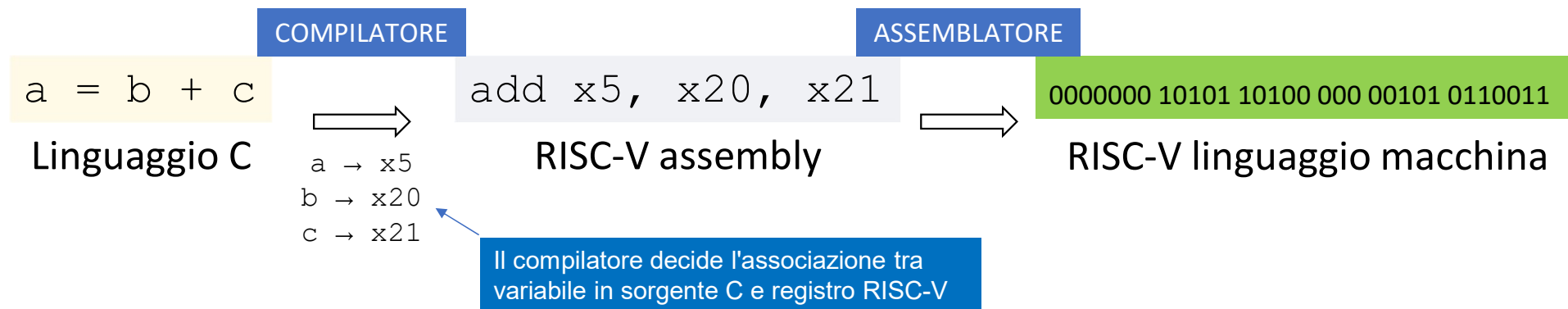


`sub a, b, c`

RISC-V assembly

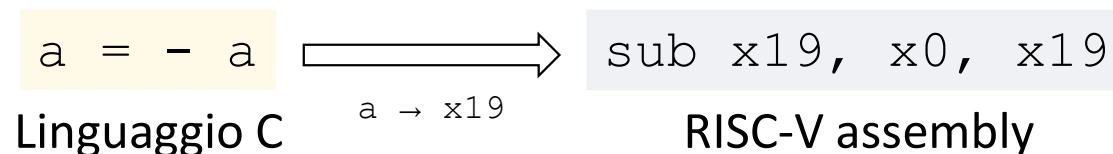
Istruzioni aritmetiche

- Operandi
 - Gli operandi devono essere sempre tre registri scelti tra x0, x1,..x31
 - La ALU ha come input solo il contenuto di registri
 - I numeri interi sono rappresentati in complemento a due (Fondamenti di Informatica, I semestre)



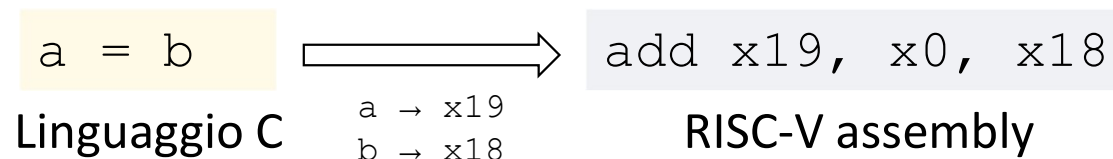
Istruzioni aritmetiche

- A volte si ha la necessità di cambiare segno al valore di un registro
- L'istruzione `sub` può essere utilizzata, memorizzando il valore 0 nel secondo operando
- In RISC-V il registro **x0** contiene sempre il valore 0



Istruzioni aritmetiche

- A volte si ha la necessità di copiare il valore di un registro in un altro
- L'istruzione `add` può essere utilizzata, usando il valore 0 per il secondo operando
- In RISC-V il registro **x0** contiene sempre il valore 0

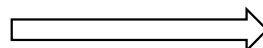


Istruzioni aritmetiche

- Il processo di traduzione di codice ad alto livello in linguaggio assembly è svolto dal compilatore
- Un'unica istruzione in un linguaggio ad alto livello può corrispondere ad una sequenza di una o più istruzioni assembly

```
f = a + b - c
```

Linguaggio C



```
f → x19  
a → x20  
b → x21  
c → x22
```

```
add x19, x20, x21  
sub x19, x19, x22
```

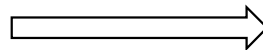
RISC-V assembly

Istruzioni aritmetiche

- Il processo di traduzione di codice ad alto livello in linguaggio assembly è svolto dal compilatore
- Un'unica istruzione in un linguaggio ad alto livello può corrispondere ad una sequenza di una o più istruzioni assembly

```
f = (g + h) - (i + j)
```

Linguaggio C



```
f → x19  
g → x20  
h → x21  
i → x22  
j → x23
```

RISC-V Instruction Set

?

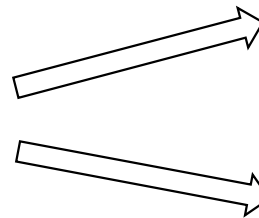
RISC-V assembly

Istruzioni aritmetiche

- Il processo di traduzione di codice ad alto livello in linguaggio assembly è svolto dal compilatore
- Un'unica istruzione in un linguaggio ad alto livello può corrispondere ad una sequenza di una o più istruzioni assembly

```
f = (g + h) - (i + j)
```

Linguaggio C



```
f → x19  
g → x20  
h → x21  
i → x22  
j → x23
```

RISC-V Instruction Set

```
add x5, x20, x21  
add x6, x22, x23  
sub x19, x5, x6
```

oppure

```
add x19, x20, x21  
sub x19, x19, x22  
sub x19, x19, x23
```

RISC-V assembly

Istruzioni di accesso alla memoria

- Che cosa accade quando
 - le variabili utilizzate in un programma sono maggiori del numero di registri a disposizione?
 - si utilizzano strutture dati complesse (vettori, liste, ecc.)?
- I dati sono salvati in memoria centrale
- La memoria centrale può essere astratta come un grande vettore monodimensionale
- Nell'esempio, la **terza** cella di memoria ha valore **00111010**
 - $M[2] = 00111010$

FONDAMENTALE:

2 è l'indirizzo del byte e **00111010** è il suo valore

Memoria

10100001	5
00000000	4
10111011	3
00111010	2
10111011	1
11111110	0

Istruzioni di accesso alla memoria

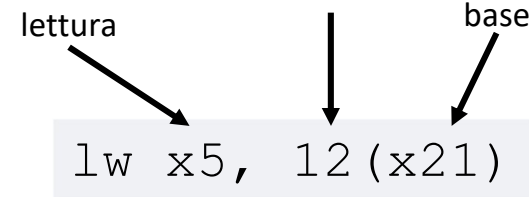
- La ALU può leggere e scrivere solo dai registri
- L'accesso alla memoria è più lento rispetto a quello dei registri
- Il compilatore si occupa di individuare la strategia più efficiente per le operazioni di caricamento e salvataggio dei dati tra registri e memoria
- Variabili utilizzate più di frequente devono rimanere il più possibile salvate nei registri

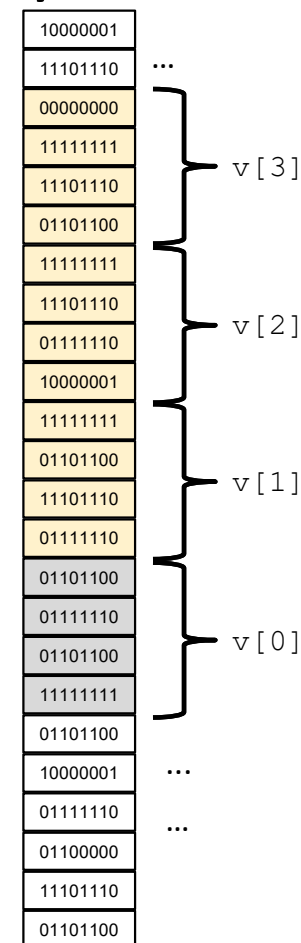
Memoria

10100001	5
00000000	4
10111011	3
00111010	2
10111011	1
11111110	0

Istruzioni di accesso alla memoria (RV32I)

- L'istruzione `load` copia un dato dalla memoria ad un registro
- L'indirizzo del dato in memoria viene specificato da:
 - Indirizzo base (contenuto in un registro)
 - Scostamento o offset (compreso tra -2048 e +2047)
- L'istruzione **`lw`** (`load word`) copia una **parola** dalla memoria ad un registro

Destinazione della lettura Offset Indirizzo base

`lw x5, 12 (x21)`



Istruzioni di accesso alla memoria (RV32I)

L'istruzione **lw** (load word) copia una **parola** dalla memoria ad un registro

Linguaggio C

```
int a;  
int v[10];  
...  
a = v[3]
```

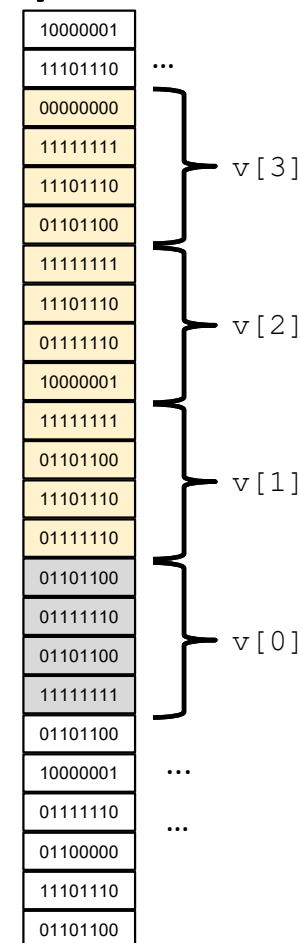
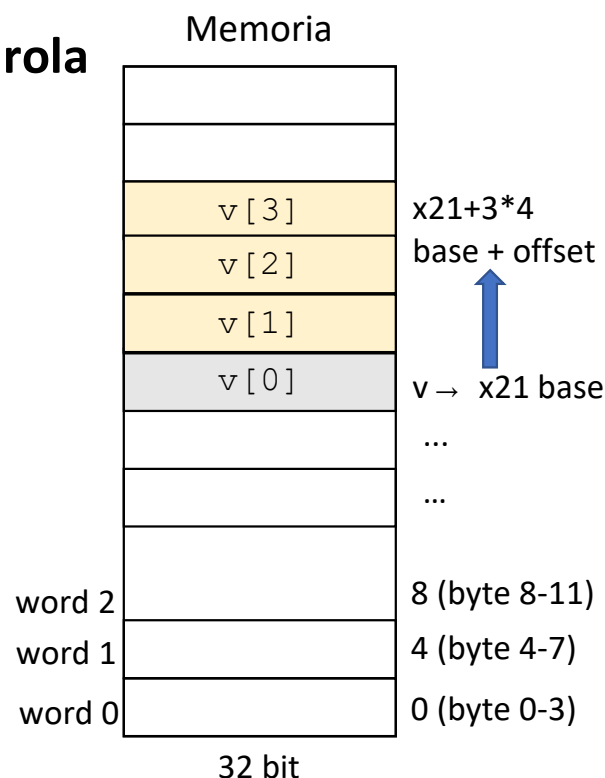
In C il tipo **int** è rappresentato con 32 bit

→
a → x5
v → x21

Destinazione della lettura Offset Indirizzo base

lw x5, 12(x21)

RISC-V assembly



Istruzioni di accesso alla memoria (RV32I)

L'istruzione **lw** (load word) copia una **parola** dalla memoria ad un registro

Linguaggio C

```
int a;  
int v[10];  
...  
a = v[3]
```

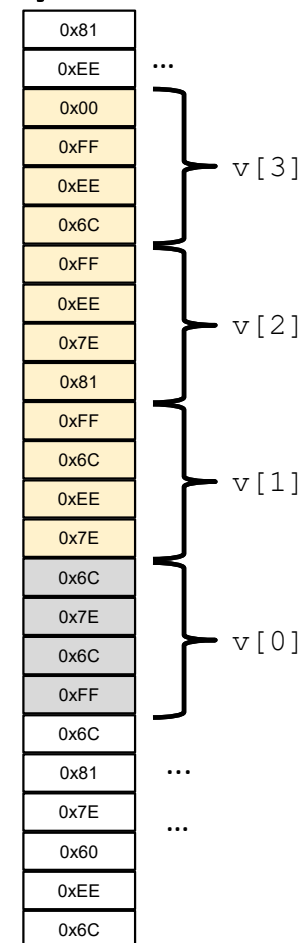
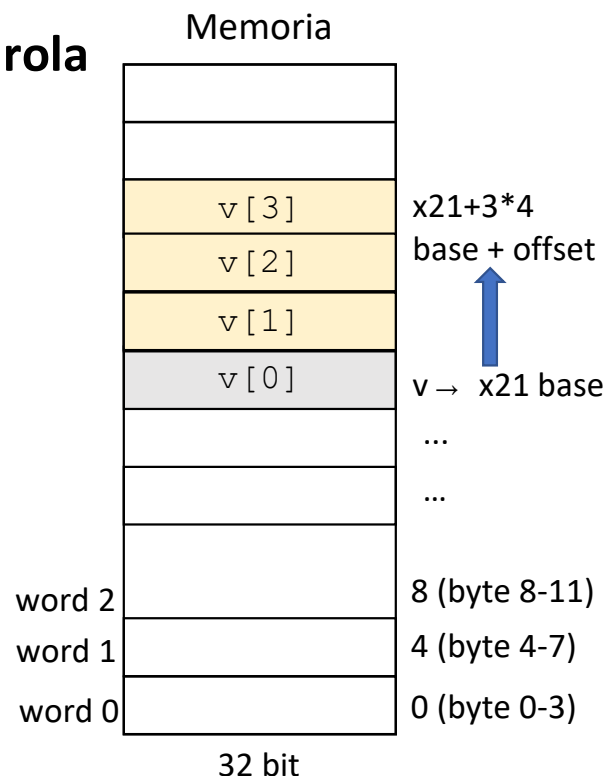
In C il tipo **int** è rappresentato con 32 bit

→
a → x5
v → x21

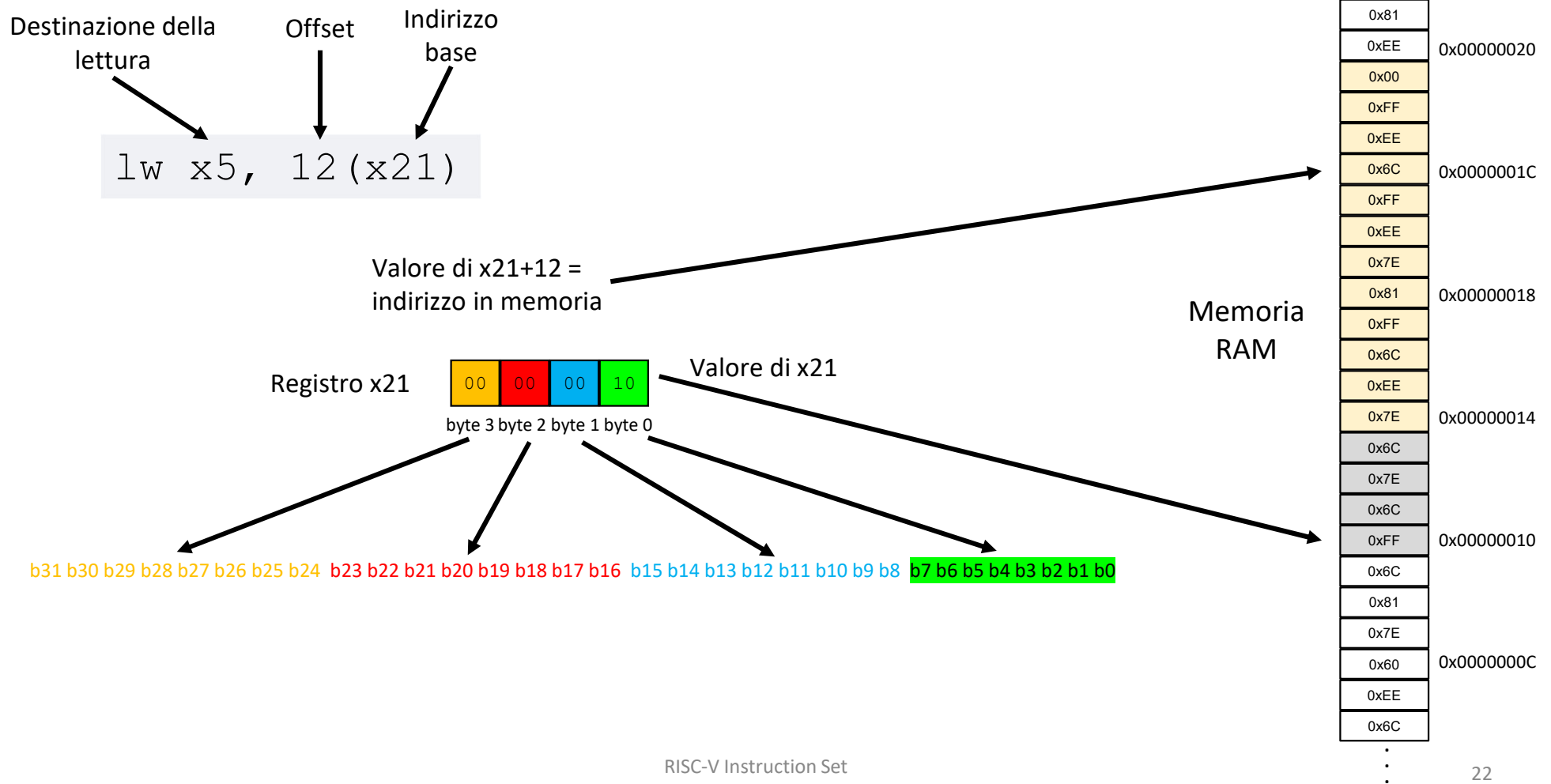
Destinazione della lettura Offset Indirizzo base

lw x5, 12(x21)

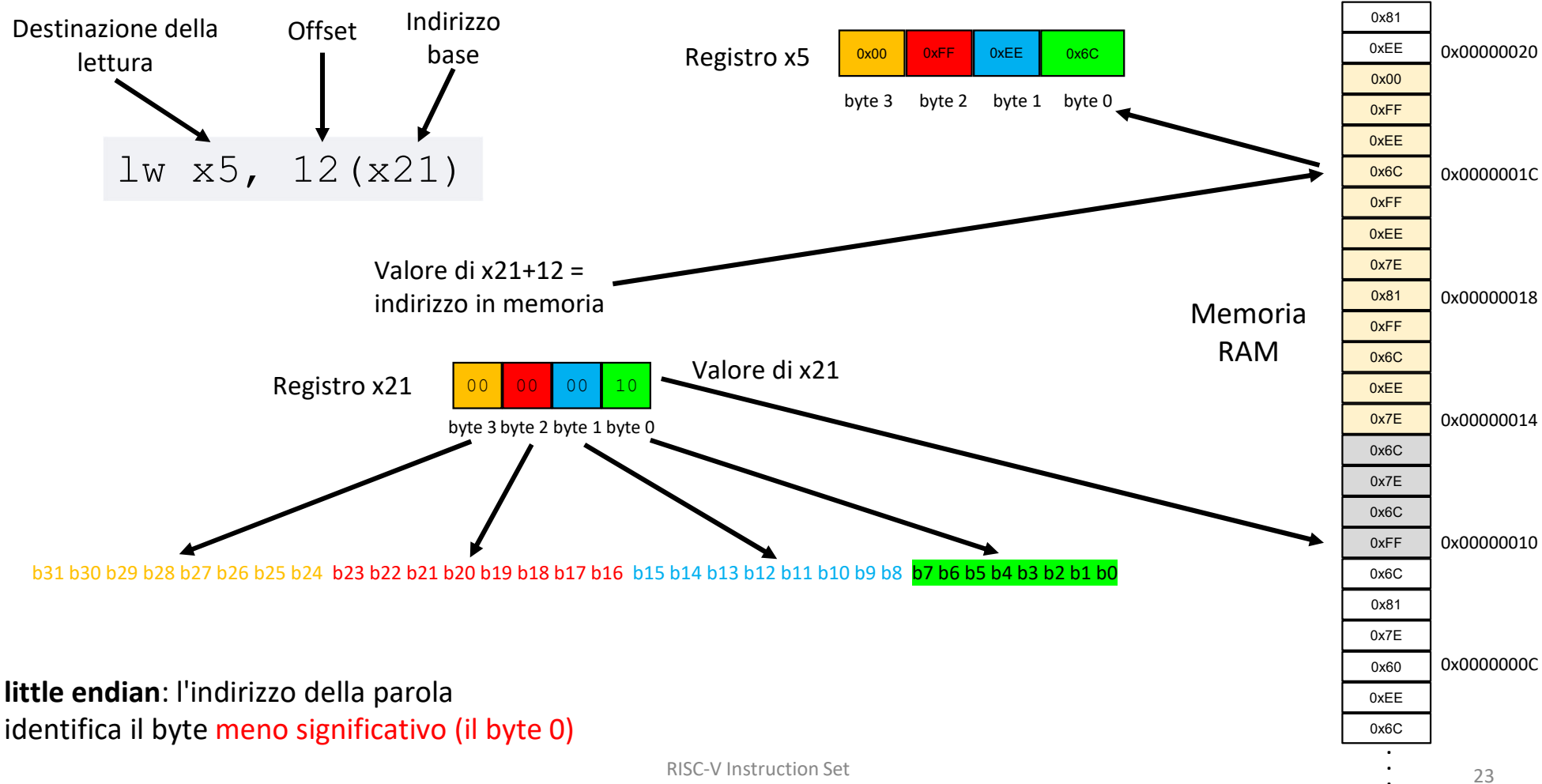
RISC-V assembly



Istruzioni di accesso alla memoria (RV32I) :



Istruzioni di accesso alla memoria (RV32I) :

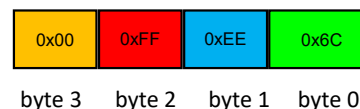


Istruzioni di accesso alla memoria (RV32I) :

Destinazione della lettura Offset Indirizzo base

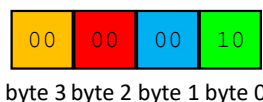
`lw x5, 12(x21)`

Registro x5



Valore di $x21+12 =$
indirizzo in memoria

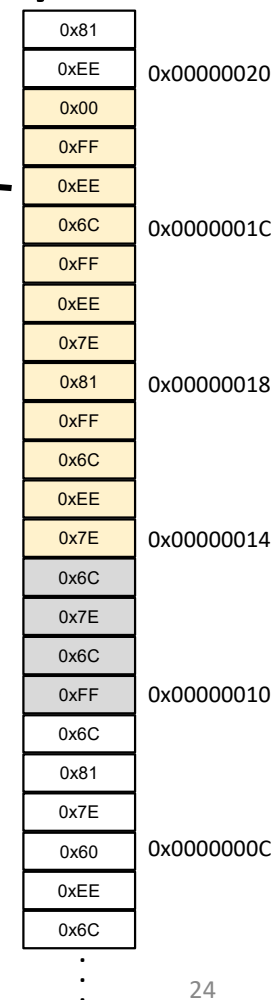
Registro x21



Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

Memoria
RAM



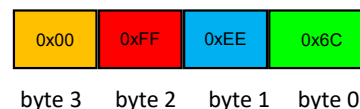
little endian: l'indirizzo della parola
identifica il byte **meno significativo (il byte 0)**

Istruzioni di accesso alla memoria (RV32I) :

Destinazione della lettura Offset Indirizzo base

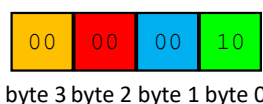
`lw x5, 12(x21)`

Registro x5



Valore di $x21+12 =$
indirizzo in memoria

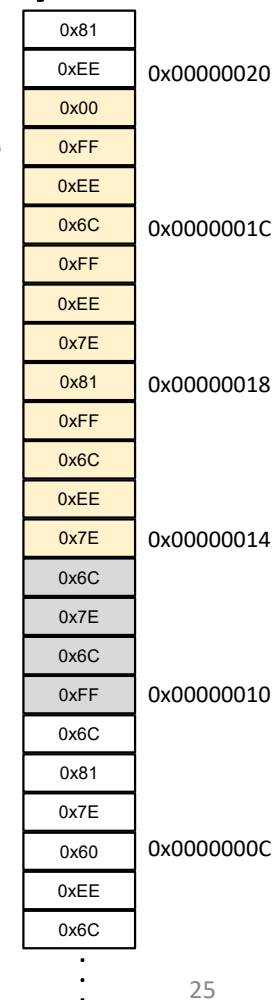
Registro x21



Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

Memoria
RAM



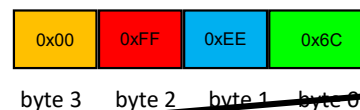
little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)

Istruzioni di accesso alla memoria (RV32I) :

Destinazione della lettura Offset Indirizzo base

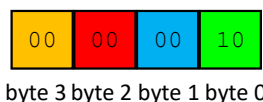
`lw x5, 12(x21)`

Registro x5



Valore di $x21+12 =$
indirizzo in memoria

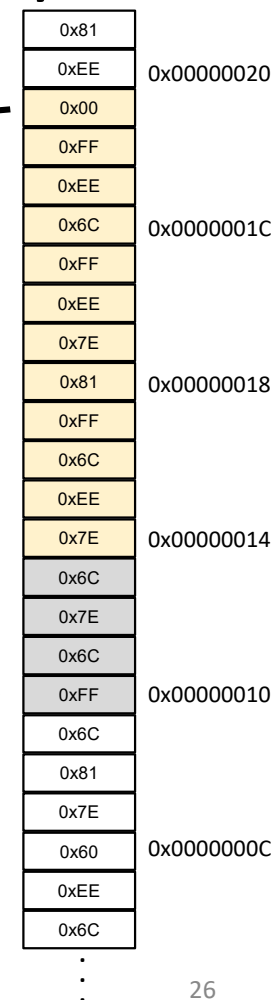
Registro x21



Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

Memoria
RAM



little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)

Istruzioni di accesso alla memoria (RV64I)

- L'istruzione `load` copia un dato dalla memoria ad un registro
- L'indirizzo del dato in memoria viene specificato da:
 - Indirizzo base (contenuto in un registro)
 - Scostamento o offset (compreso tra -2048 e +2047)
- L'istruzione **`ld`** (`load doubleword`) copia una **parola doppia** dalla memoria ad un registro

In C il tipo **long int** è rappresentato con 64 bit

Linguaggio C

```
long int a;  
long int v[10];  
...  
a = v[3]
```

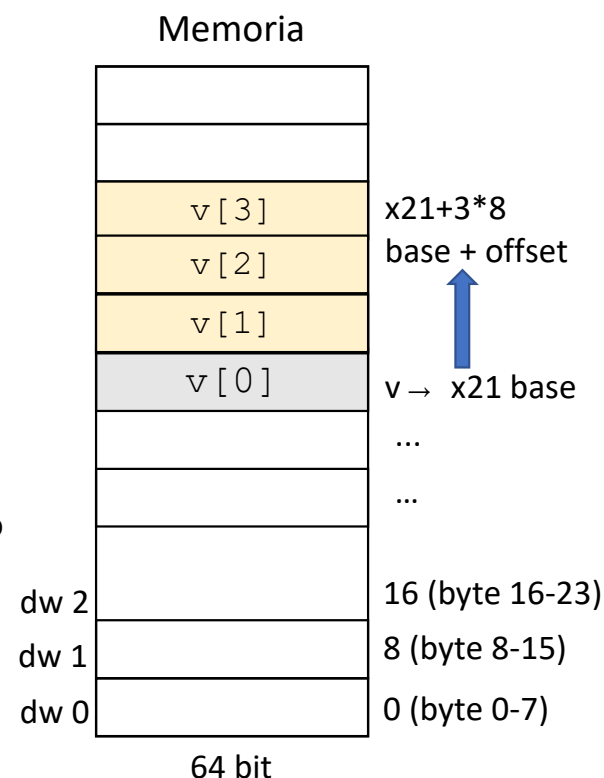
→
a → x5
v → x21

Destinazione della lettura Offset Indirizzo base

↓ ↓ ↓

`ld x5, 24(x21)`

RISC-V assembly



FONDAMENTALE: `ld` e `sd` sono disponibili nell'architettura a 64 bit (ma non a 32 bit)

Istruzioni di accesso alla memoria (RV32I)

- L'istruzione `store` copia un dato da un registro alla memoria
- L'indirizzo di destinazione in memoria viene specificato da:
 - Indirizzo base (contenuto in un registro)
 - Scostamento o offset (compreso tra -2048 e +2047)
- L'istruzione **sw** (`store word`) copia una **parola** in memoria

In C il tipo `int` è rappresentato con 32 bit

Linguaggio C

```
int a;  
int v[10];  
...  
v[2] = a
```

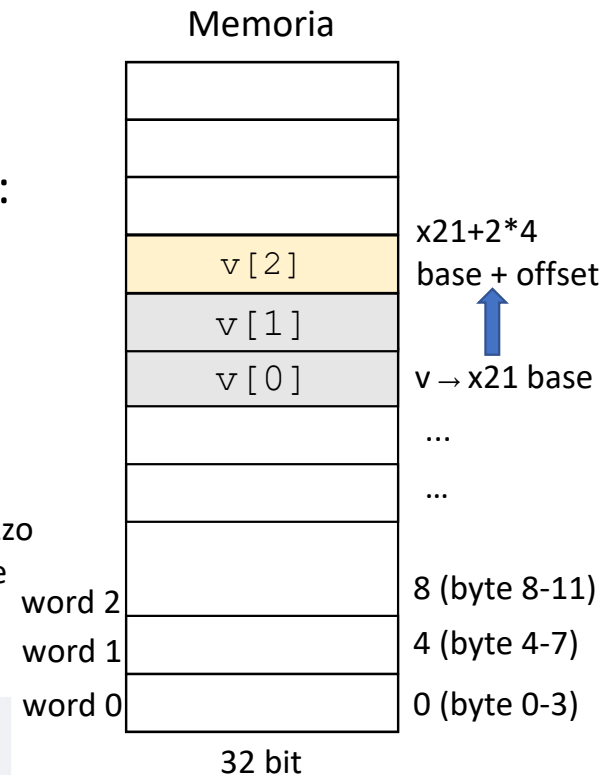
Dato da scrivere in memoria Offset Indirizzo base

→

`sw x5, 8(x21)`

$a \rightarrow x5$
 $v \rightarrow x21$

RISC-V assembly

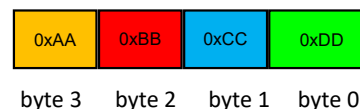


Istruzioni di accesso alla memoria (RV32I) :

Dato da scrivere in memoria Offset Indirizzo base

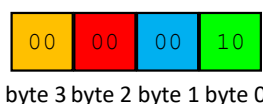
`sw x5, 8(x21)`

Registro x5



Valore di $x21+8$ =
indirizzo in memoria

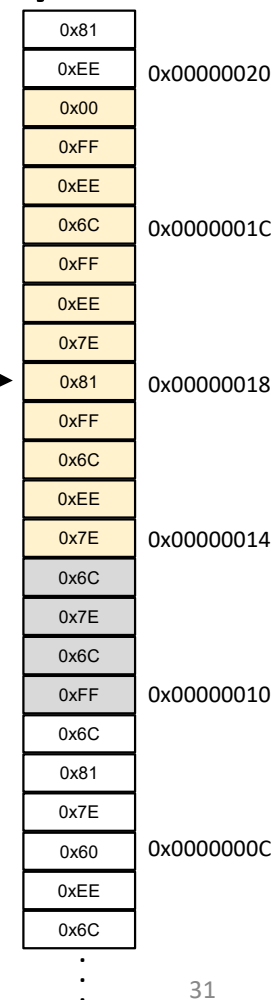
Registro x21



Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

Memoria
RAM



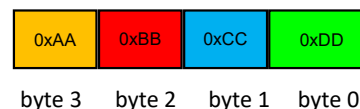
little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)

Istruzioni di accesso alla memoria (RV32I) :

Dato da scrivere in memoria Offset Indirizzo base

`sw x5, 8(x21)`

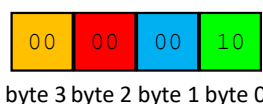
Registro x5



Valore di $x21+8$ =
indirizzo in memoria

Memoria
RAM

Registro x21

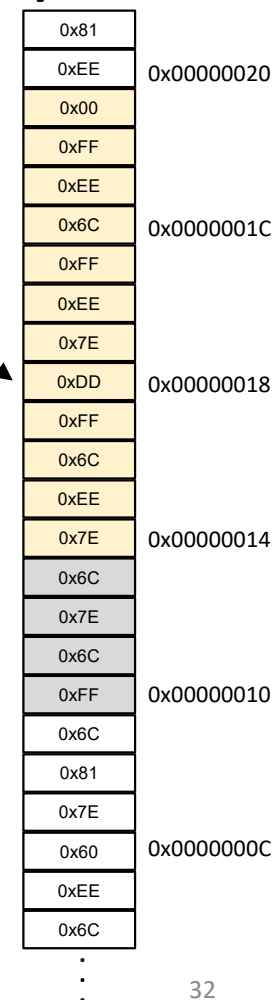


Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)

RISC-V Instruction Set

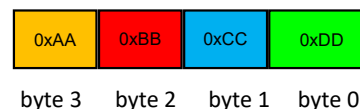


Istruzioni di accesso alla memoria (RV32I) :

Dato da scrivere in memoria Offset Indirizzo base

`sw x5, 8(x21)`

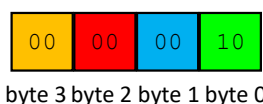
Registro x5



Valore di $x21+8$ =
indirizzo in memoria

Memoria
RAM

Registro x21

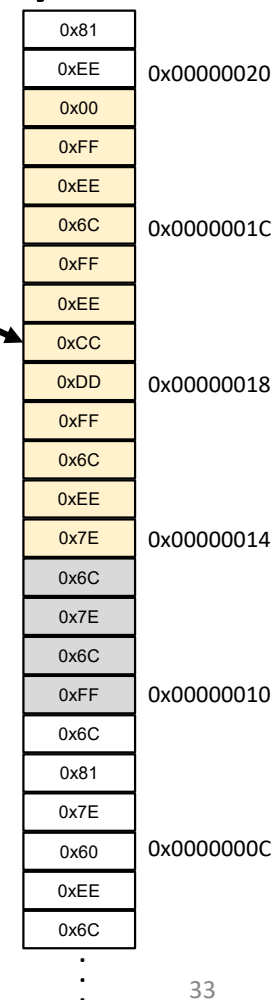


Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)

RISC-V Instruction Set

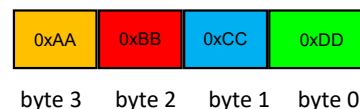


Istruzioni di accesso alla memoria (RV32I) :

Dato da scrivere in memoria Offset Indirizzo base

`sw x5, 8(x21)`

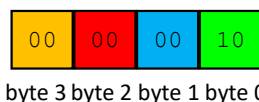
Registro x5



Valore di $x21+8$ =
indirizzo in memoria

Memoria
RAM

Registro x21

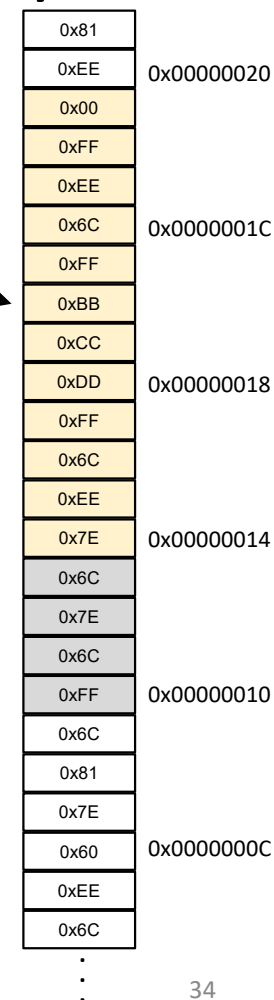


Valore di x21

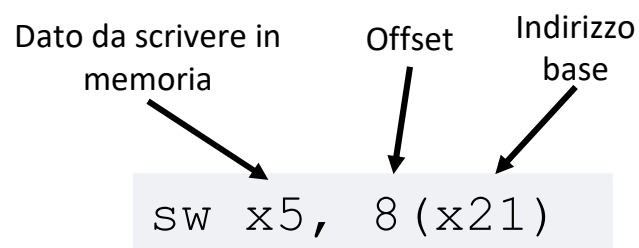
b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)

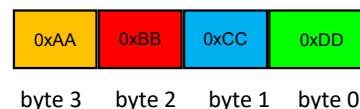
RISC-V Instruction Set



Istruzioni di accesso alla memoria (RV32I) :



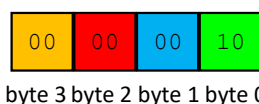
Registro x5



Valore di x21+8 =
indirizzo in memoria

Memoria
RAM

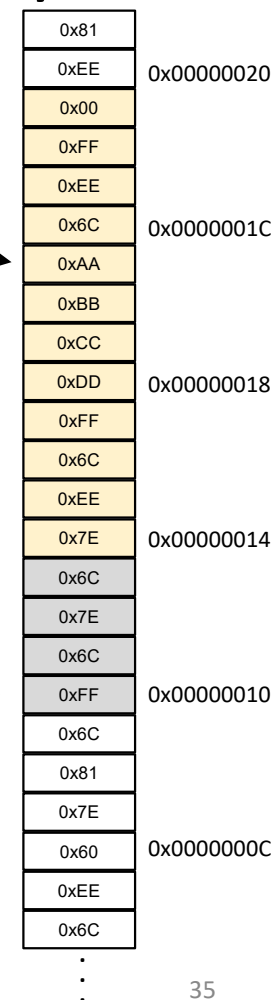
Registro x21



Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)

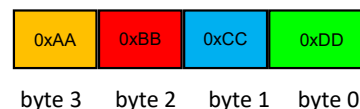


Istruzioni di accesso alla memoria (RV32I) :

Dato da scrivere in memoria Offset Indirizzo base

`sh x5, 8(x21)`

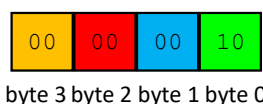
Registro x5



Valore di $x21+8$ =
indirizzo in memoria

Memoria
RAM

Registro x21

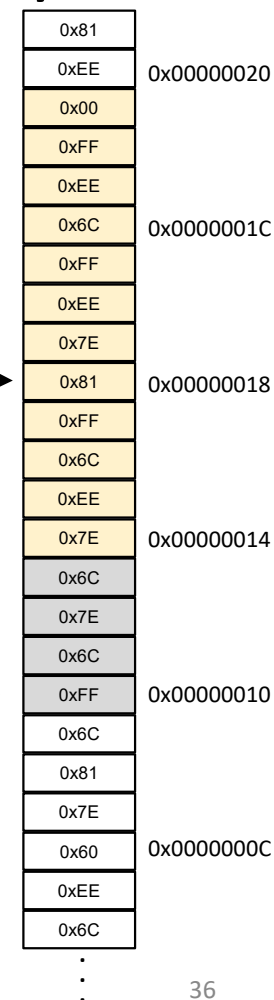


Valore di x21

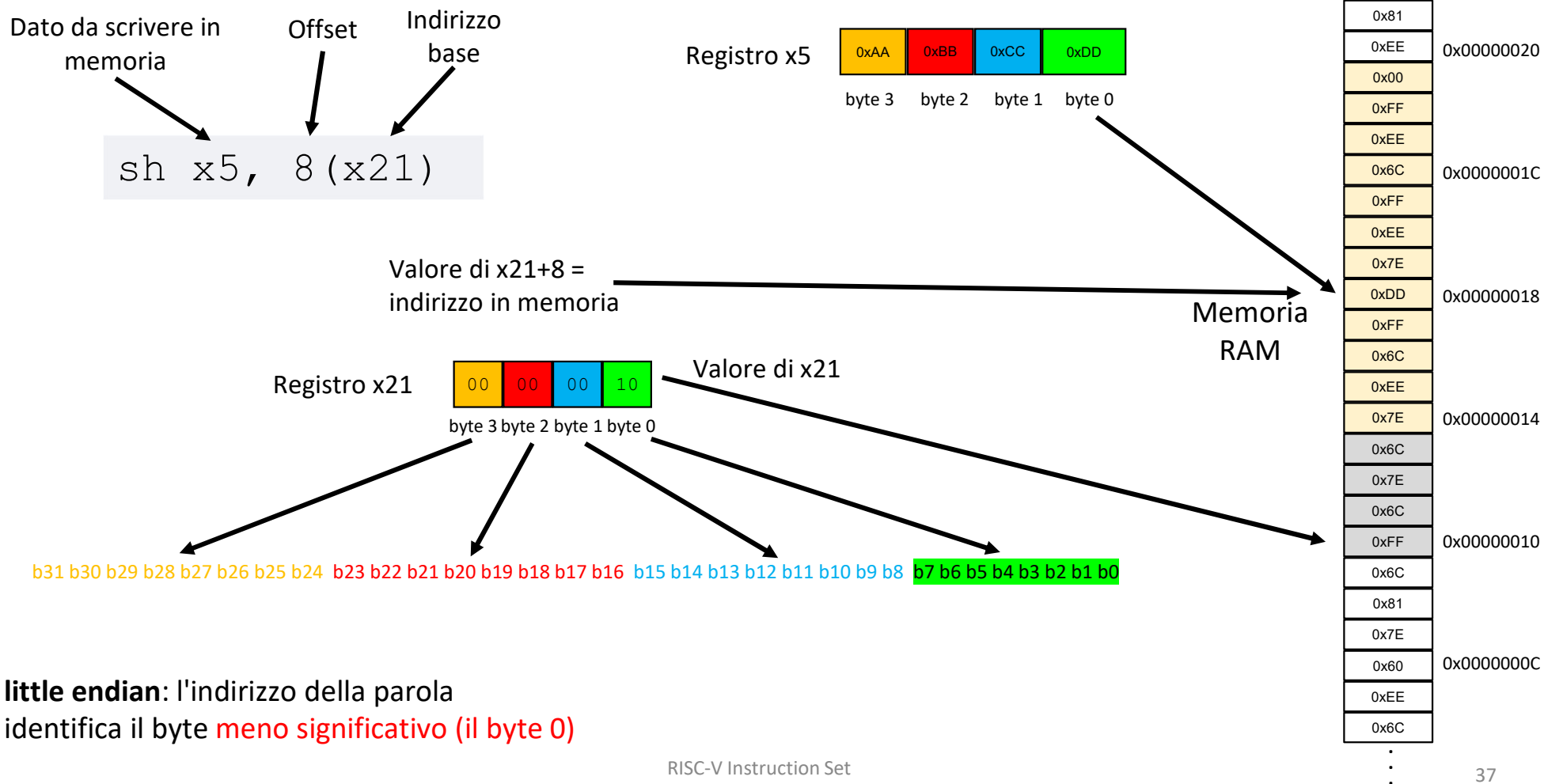
b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)

RISC-V Instruction Set



Istruzioni di accesso alla memoria (RV32I):

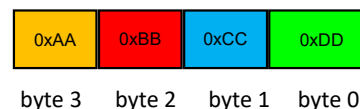


Istruzioni di accesso alla memoria (RV32I) :

Dato da scrivere in memoria Offset Indirizzo base

`sh x5, 8 (x21)`

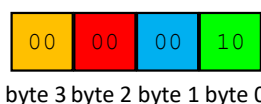
Registro x5



Valore di $x21+8$ =
indirizzo in memoria

Memoria
RAM

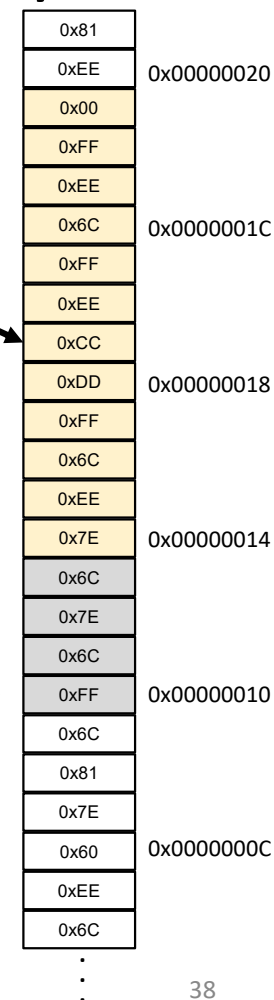
Registro x21



Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

little endian: l'indirizzo della parola
identifica il byte **meno significativo (il byte 0)**

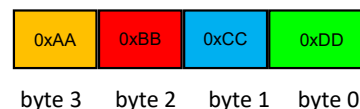


Istruzioni di accesso alla memoria (RV32I) :

Dato da scrivere in memoria Offset Indirizzo base

`sb x5, 8(x21)`

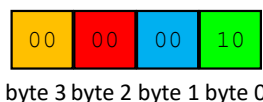
Registro x5



Valore di $x21+8$ =
indirizzo in memoria

Memoria
RAM

Registro x21

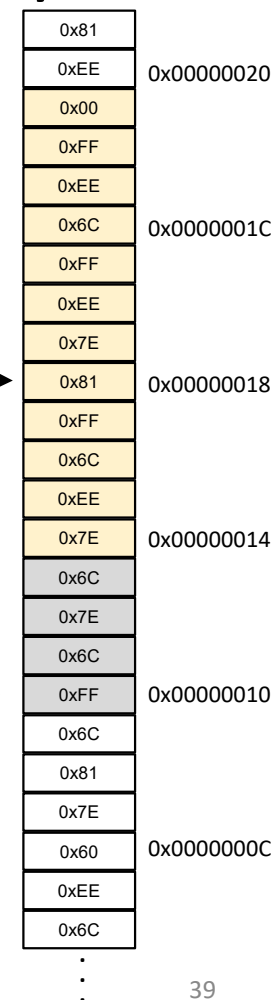


Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)

RISC-V Instruction Set

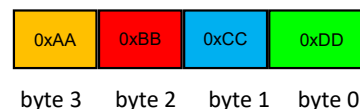


Istruzioni di accesso alla memoria (RV32I) :

Dato da scrivere in memoria Offset Indirizzo base

`sb x5, 8(x21)`

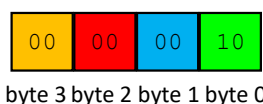
Registro x5



Valore di $x21+8$ =
indirizzo in memoria

Memoria
RAM

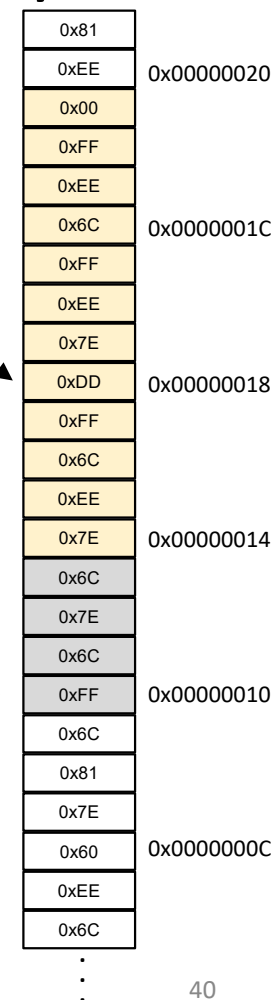
Registro x21



Valore di x21

b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

little endian: l'indirizzo della parola
identifica il byte **meno significativo (il byte 0)**

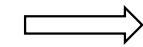


Istruzioni di accesso alla memoria (RV32I)

- Un esempio

```
int g,h,f;  
int v[10];  
...  
g = h + v[3]  
v[6] = g - f
```

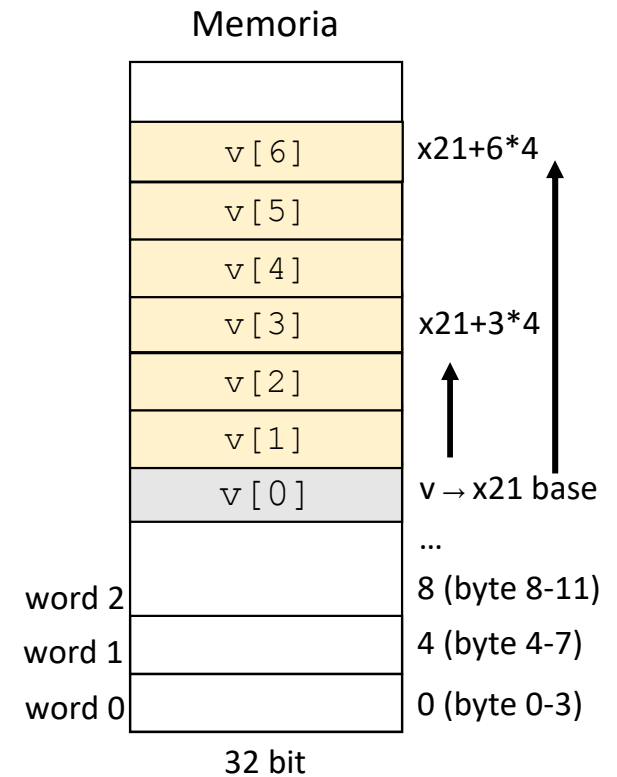
Linguaggio C



$g \rightarrow x5$
 $h \rightarrow x9$
 $v \rightarrow x21$
 $f \rightarrow x19$



RISC-V assembly



Istruzioni di accesso alla memoria (RV32I)

- Un esempio

```
int g,h,f;  
int v[10];  
...
```

```
g = h + v[3]  
v[6] = g - f
```

Linguaggio C

$g \rightarrow x5$
 $h \rightarrow x9$
 $v \rightarrow x21$
 $f \rightarrow x19$

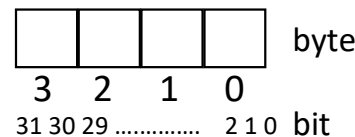
registro base

offset

```
lw x10, 12(x21)  
add x5, x9, x10  
sub x6, x5, x19  
sw x6, 24(x21)
```

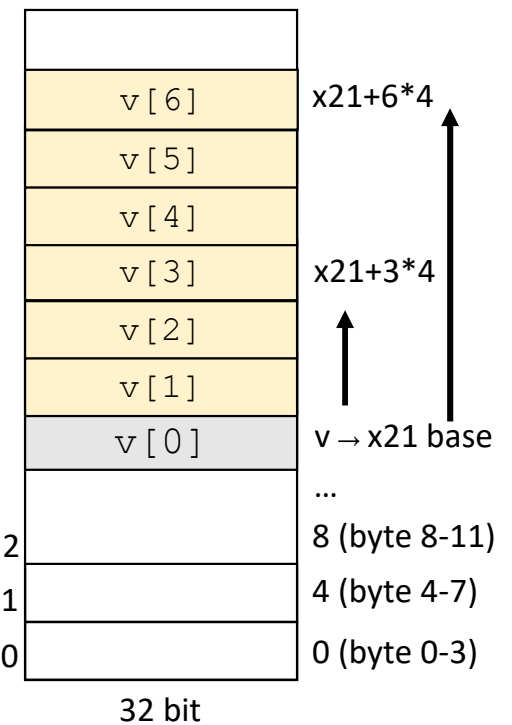
RISC-V assembly

little endian: l'indirizzo della parola
identifica il byte **meno significativo** (il byte 0)



RISC-V Instruction Set

Memoria



Istruzioni di accesso alla memoria (RV64I)

- L'istruzione `store` copia un dato da un registro alla memoria
- L'indirizzo di destinazione in memoria viene specificato da:
 - Indirizzo base (contenuto in un registro)
 - Scostamento o offset (compreso tra -2048 e +2047)
- L'istruzione **`sd`** (`store doubleword`) copia una **parola doppia** in memoria

Linguaggio C

```
long int a;  
long int v[10];  
...  
v[2] = a
```

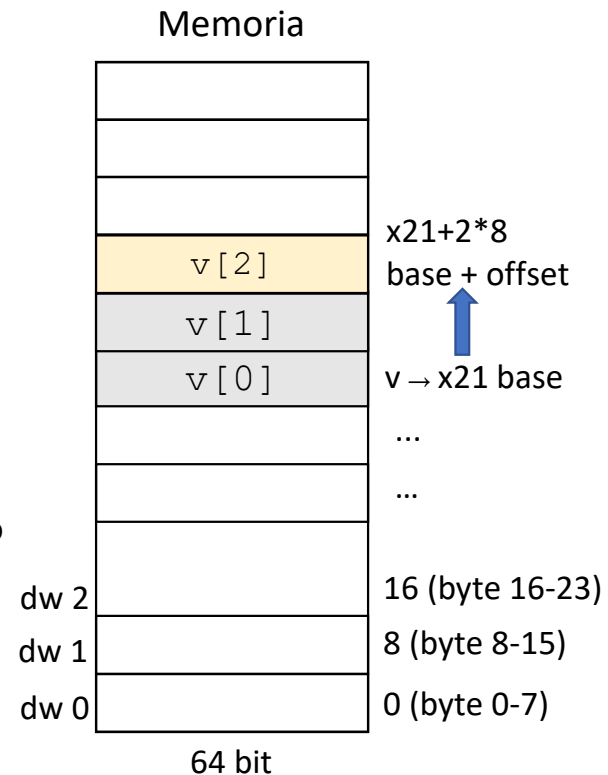
Dato da scrivere in memoria Offset Indirizzo base

→

`sd x5, 16(x21)`

RISC-V assembly

$a \rightarrow x5$
 $v \rightarrow x21$

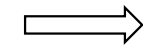


Istruzioni di accesso alla memoria (RV64I)

- Un esempio

```
long int g, h, f;  
long int v[10];  
...  
g = h + v[3]  
v[6] = g - f
```

Linguaggio C

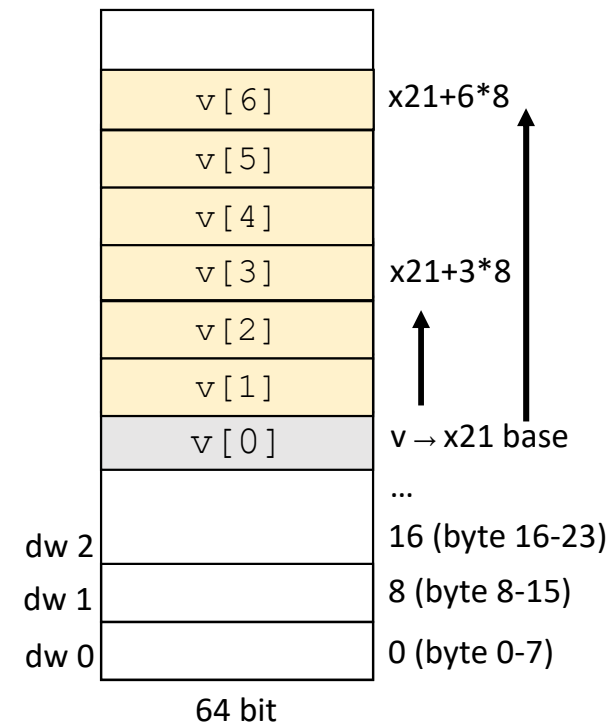


$g \rightarrow x5$
 $h \rightarrow x9$
 $v \rightarrow x21$
 $f \rightarrow x19$



RISC-V assembly

Memoria



Istruzioni di accesso alla memoria (RV64I)

- Un esempio

```
long int g, h, f;  
long int v[10];  
...  
g = h + v[3]  
v[6] = g - f
```

Linguaggio C

g → x5
h → x9
v → x21
f → x19

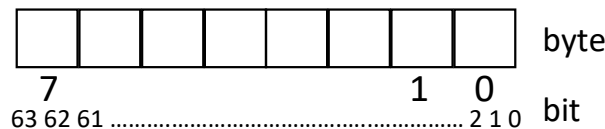
registro base

offset

```
ld x10, 24(x21)  
add x5, x9, x10  
sub x6, x5, x19  
sd x6, 48(x21)
```

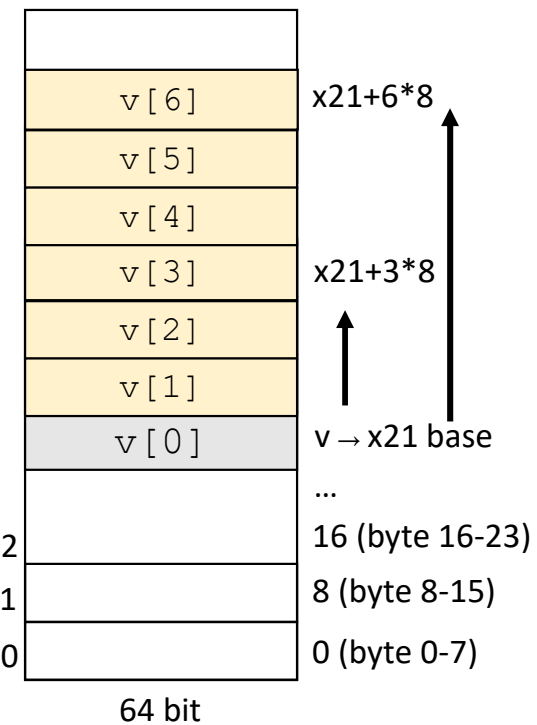
RISC-V assembly

little endian: indirizzo di (doppia) parola
identifica il byte meno significativo (il byte 0)



RISC-V Instruction Set

Memoria

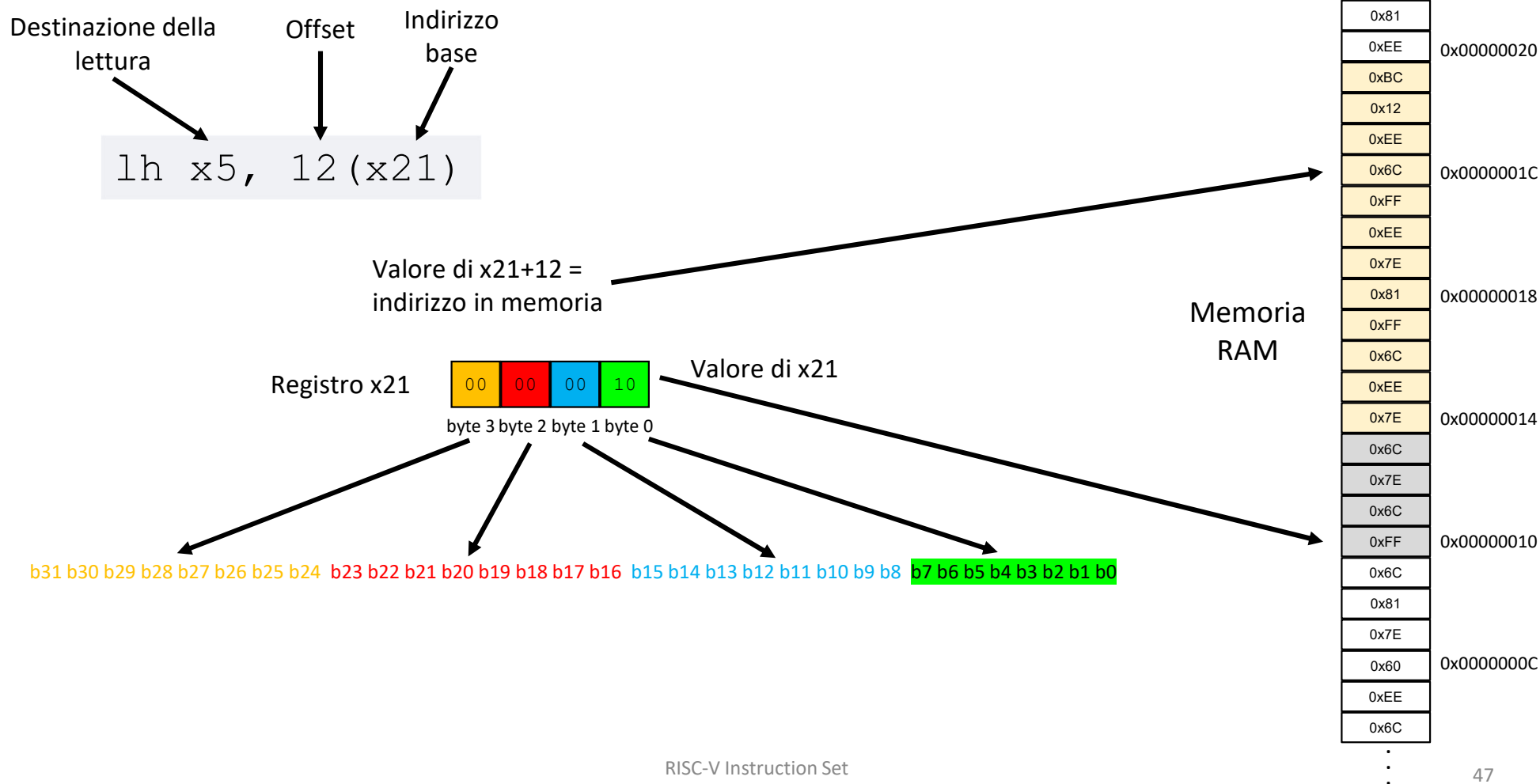


Accesso a byte, half-word e word (RV32I)

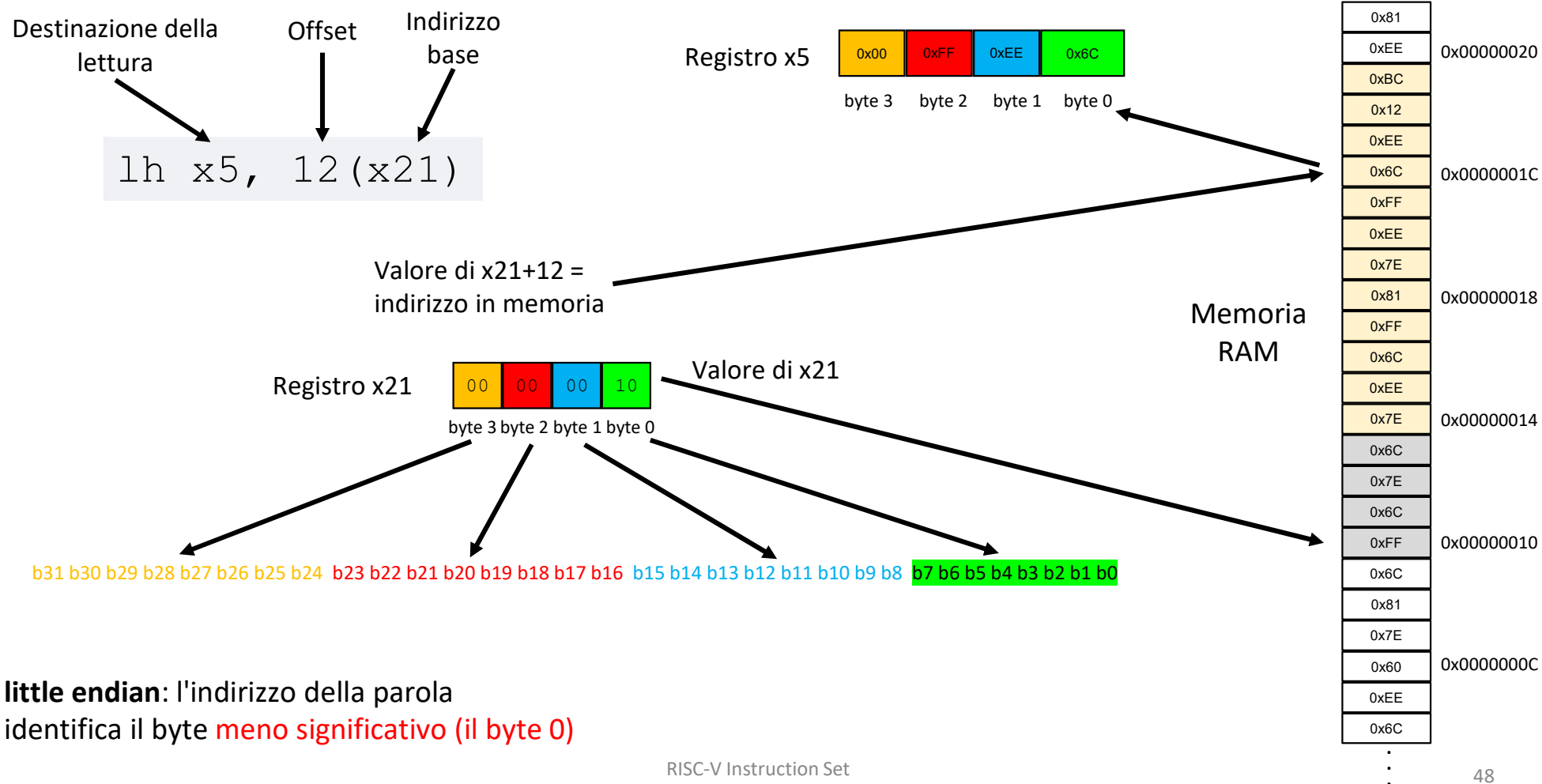
- Per accedere al singolo byte sono a disposizione
 - (Utile per le stringhe di caratteri ASCII)
 - `lb x5, 0(x6)` "load byte"
 - `sb x5, 0(x6)` "store byte"
- Per accedere alla half-word (16 bit) ci sono
 - (Utile per le stringhe di caratteri UNICODE, es. in Java)
 - `lh x5, 0(x6)` "load half-word"
 - `sh x5, 0(x6)` "store half-word"
- Per accedere alla word (32 bit) ci sono
 - `lw x5, 0(x6)` "load word"
 - `sw x5, 0(x6)` "store word"

Nota: in fase di caricamento (load), dovendo porre la quantità da 8/16 bit in 32 bit, viene automaticamente effettuata l'**estensione del segno**. Se ciò non si vuole, si devono usare `lbu` (al posto di `lb`) e `lhu` (al posto di `lh`) ed estensione con 0

Istruzioni di accesso alla memoria (RV32I) :



Istruzioni di accesso alla memoria (RV32I) :

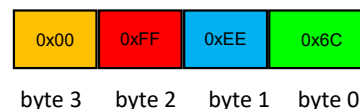


Istruzioni di accesso alla memoria (RV32I) :

Destinazione della lettura Offset Indirizzo base

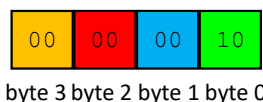
lh x5, 12(x21)

Registro x5



Valore di x21+12 =
indirizzo in memoria

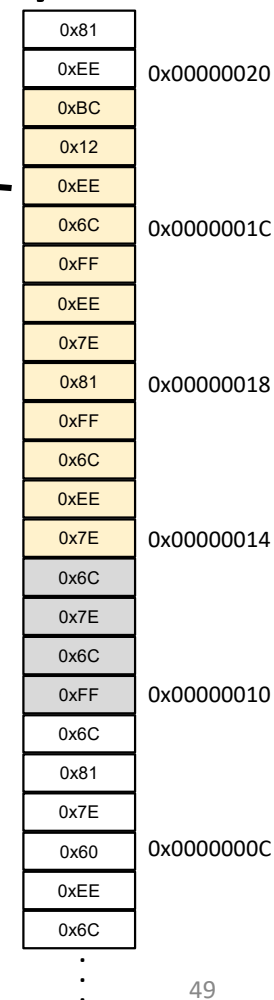
Registro x21



Valore di x21

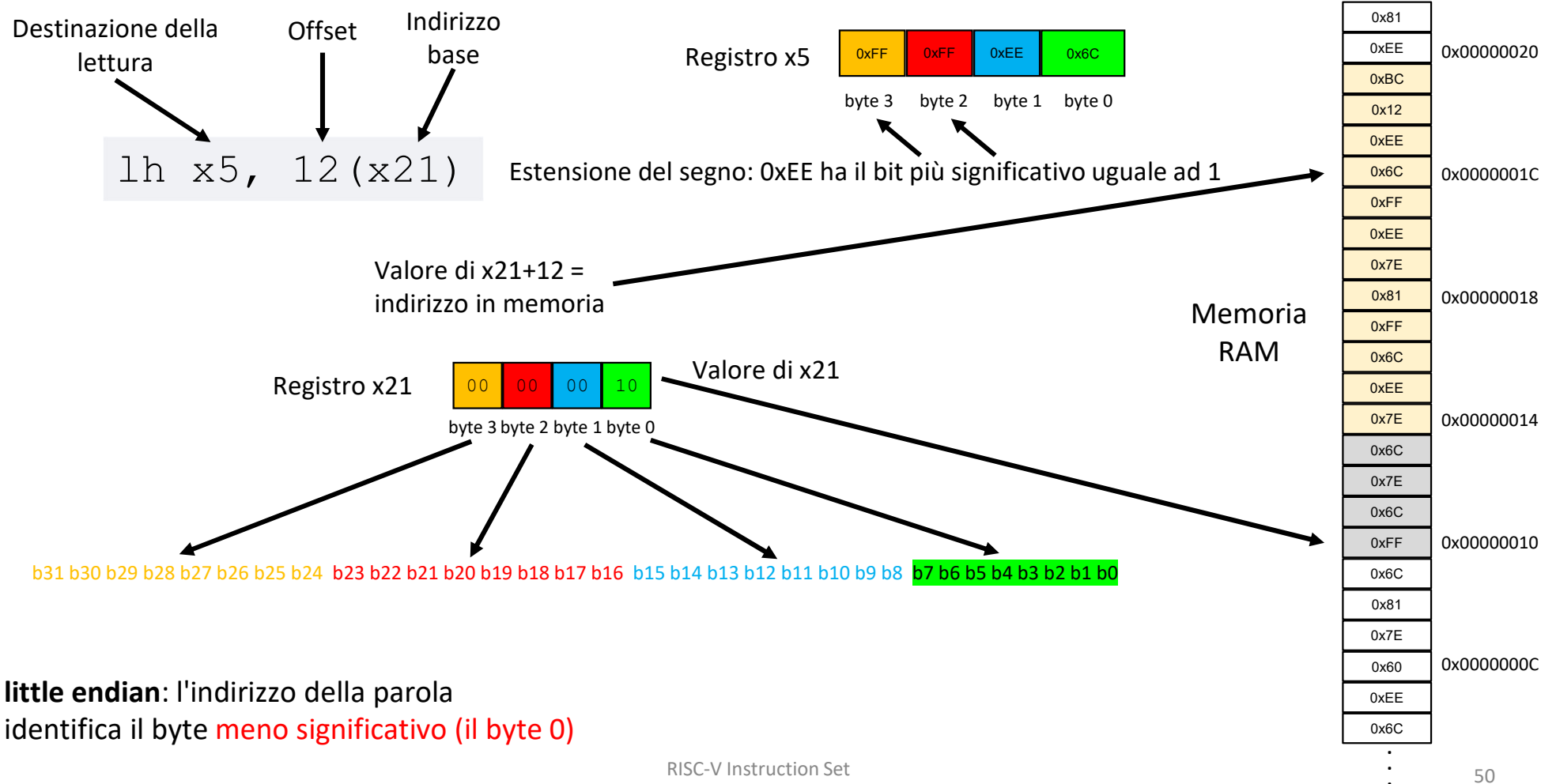
b31 b30 b29 b28 b27 b26 b25 b24 b23 b22 b21 b20 b19 b18 b17 b16 b15 b14 b13 b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0

Memoria
RAM

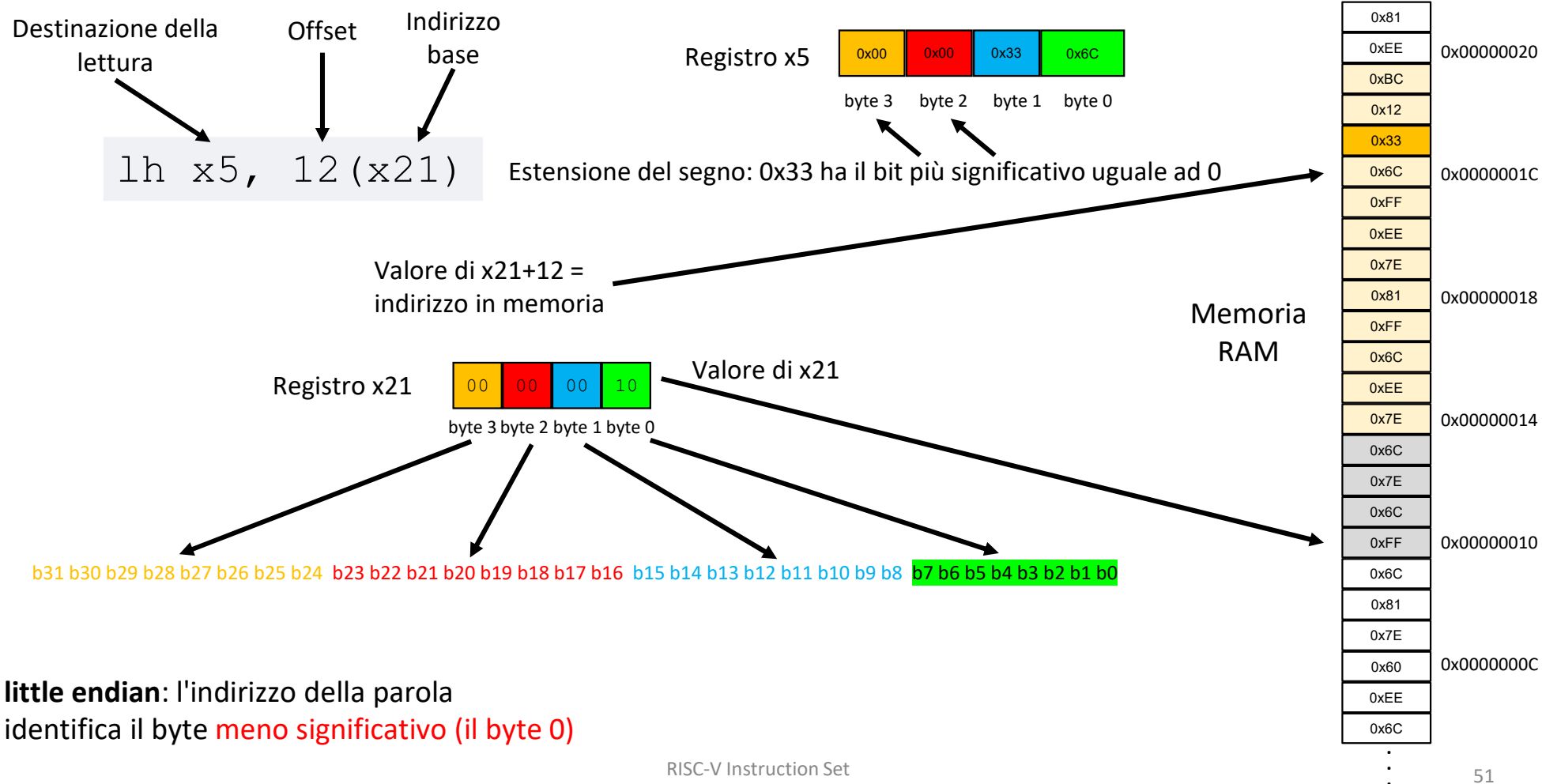


little endian: l'indirizzo della parola
identifica il byte **meno significativo (il byte 0)**

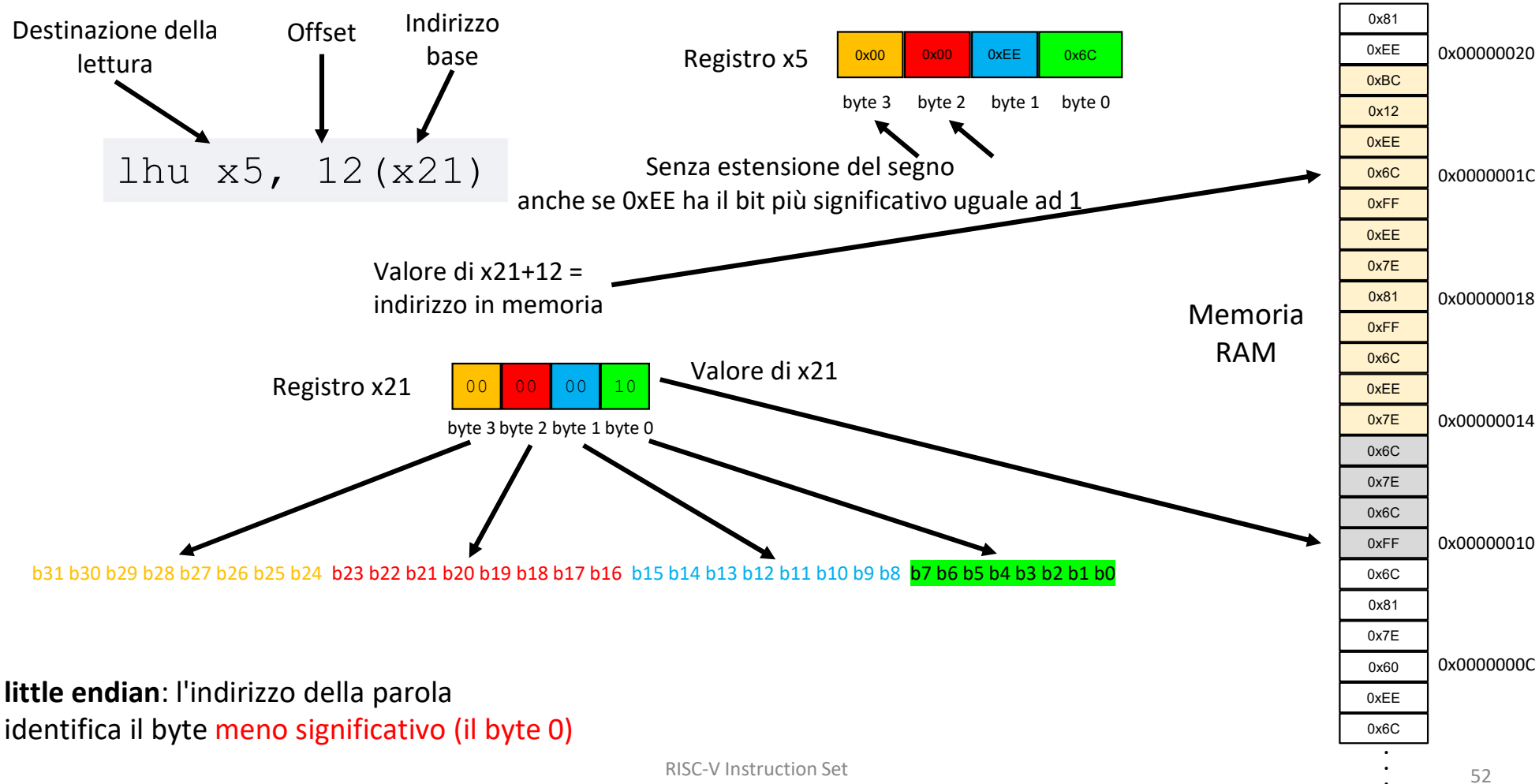
Istruzioni di accesso alla memoria (RV32I) :



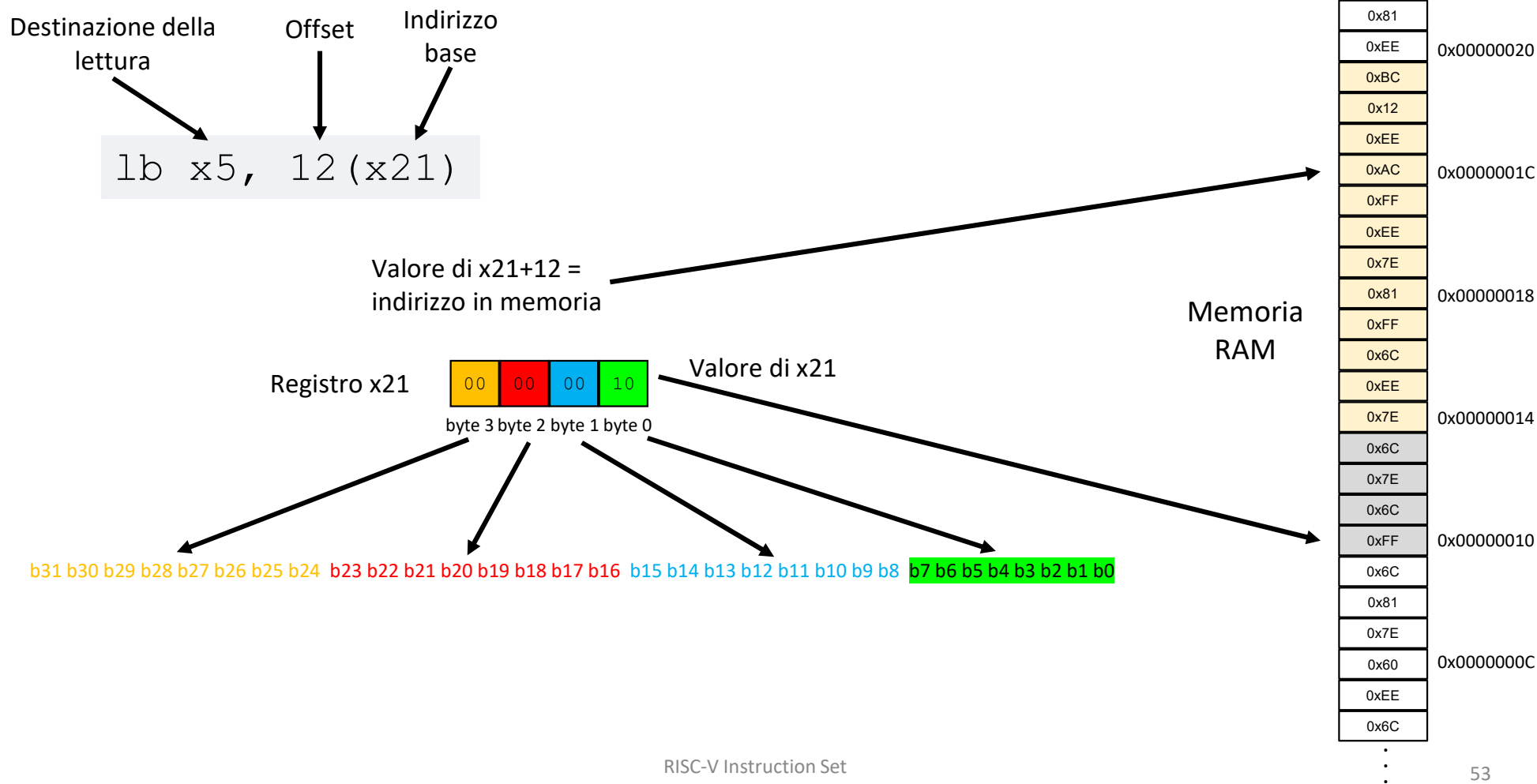
Istruzioni di accesso alla memoria (RV32I) :



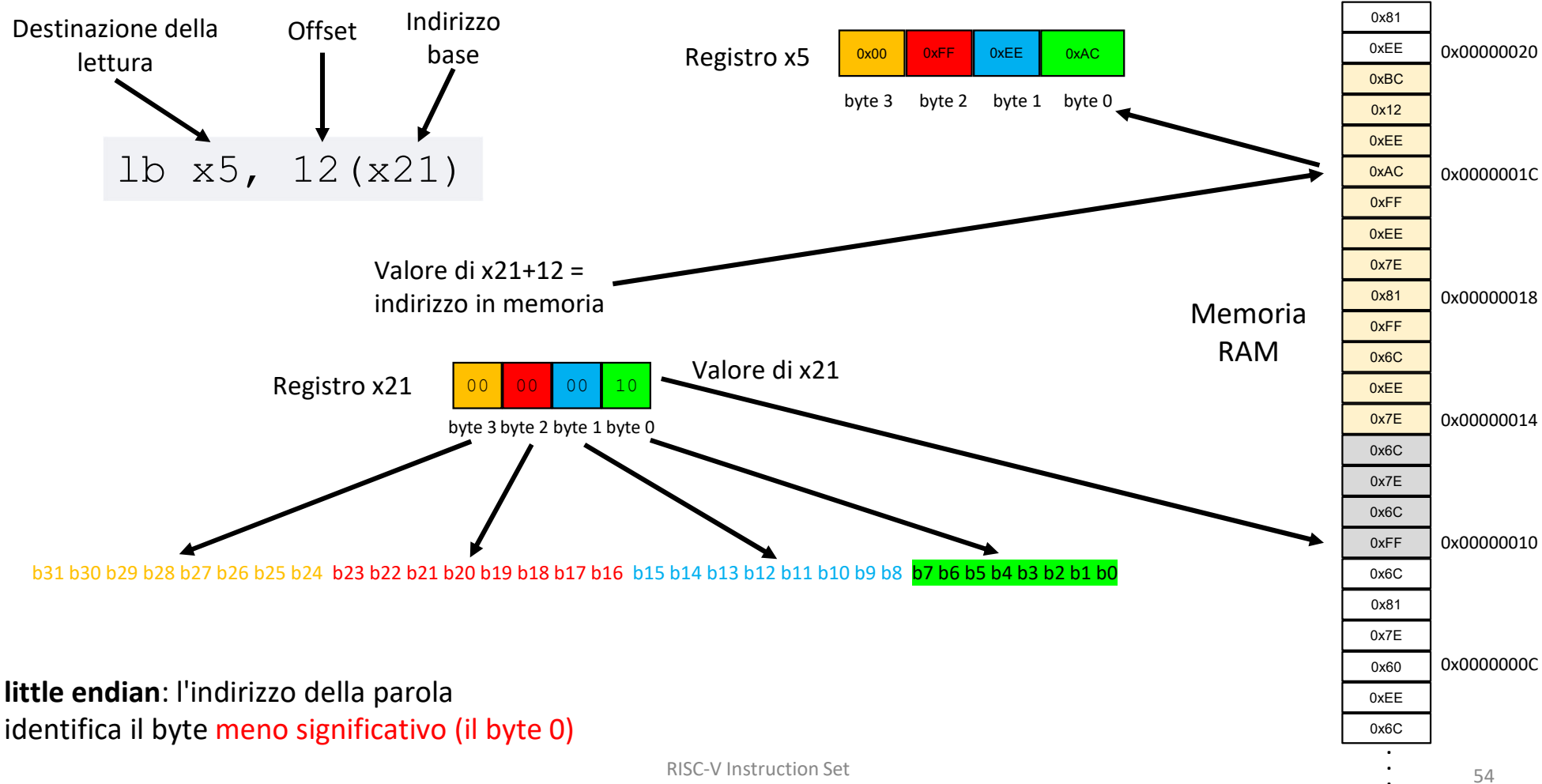
Istruzioni di accesso alla memoria (RV32I) :



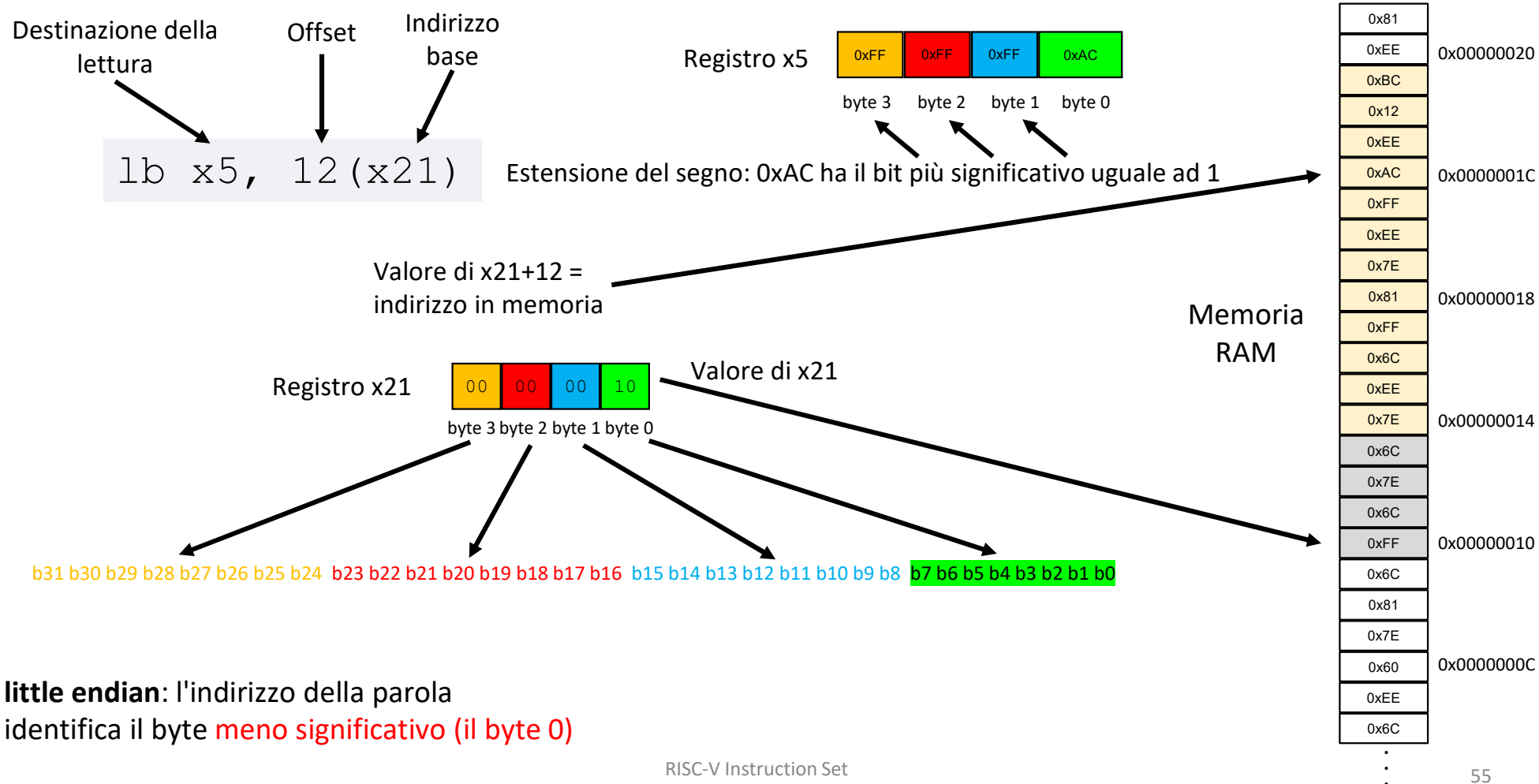
Istruzioni di accesso alla memoria (RV32I) :



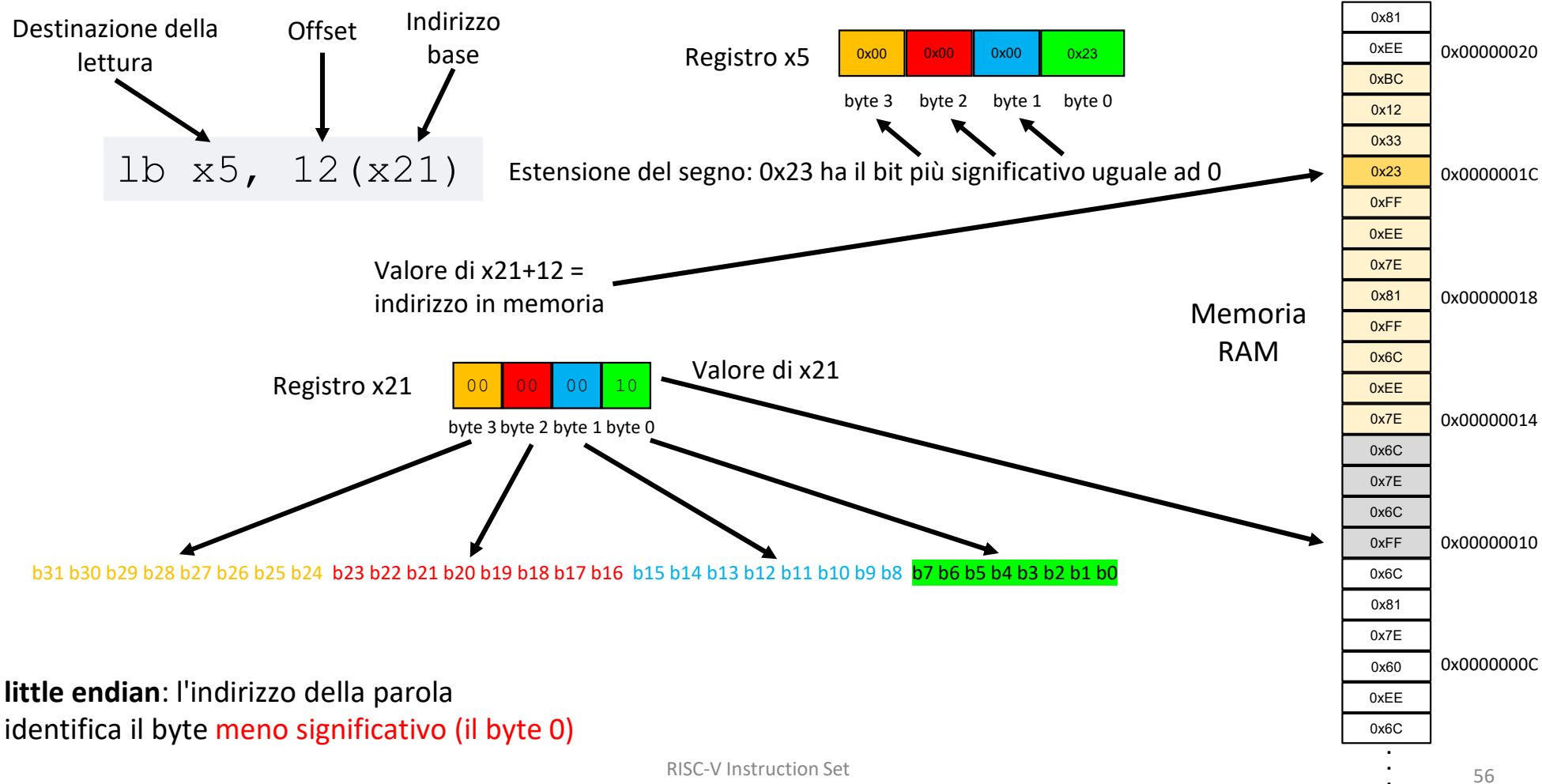
Istruzioni di accesso alla memoria (RV32I) :



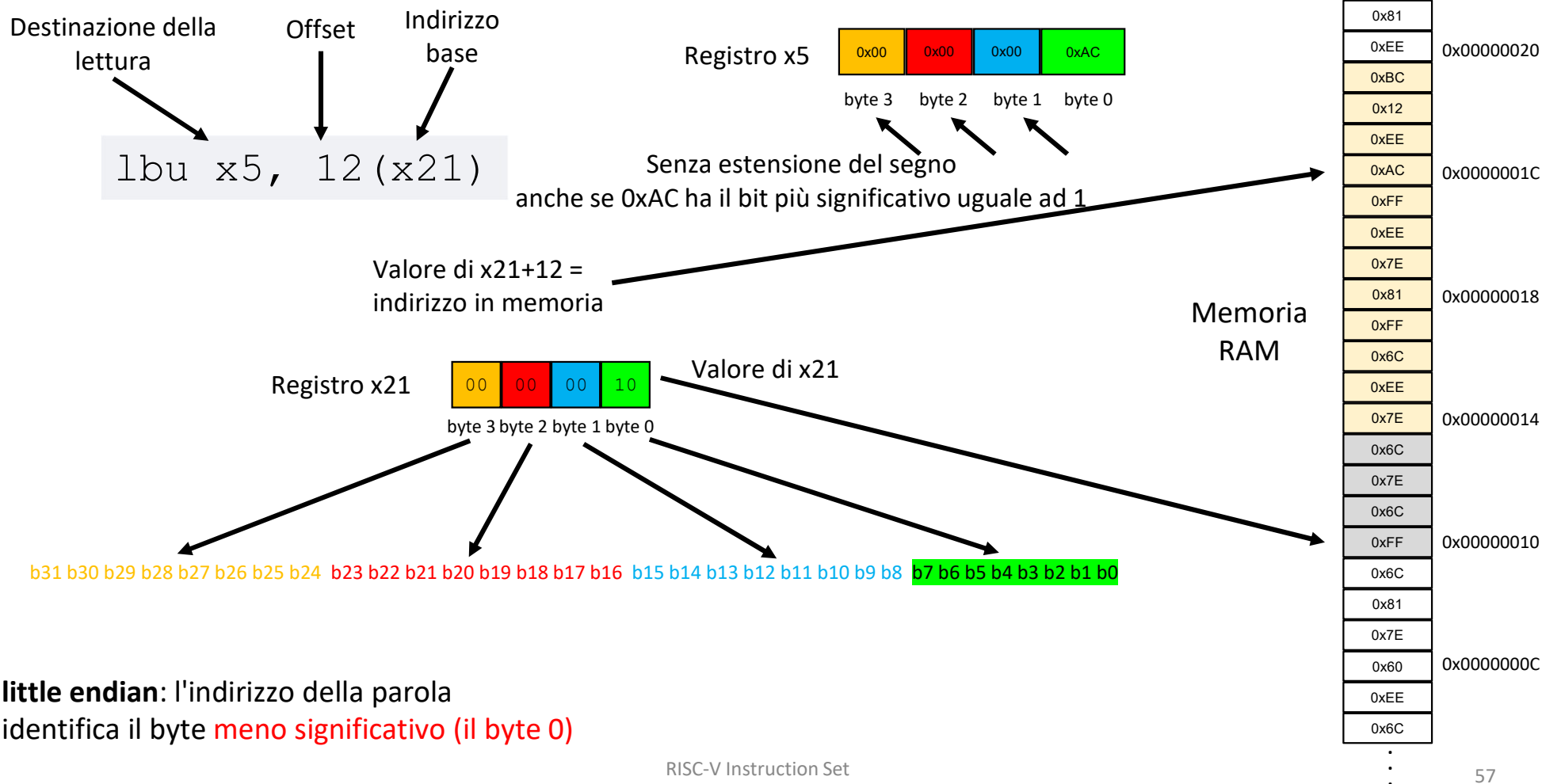
Istruzioni di accesso alla memoria (RV32I) :



Istruzioni di accesso alla memoria (RV32I) :



Istruzioni di accesso alla memoria (RV32I) :



Accesso a byte, half-word e word (RV64I)

- Per accedere al singolo byte sono a disposizione
 - (Utile per le stringhe di caratteri ASCII)
 - `lb x5, 0(x6)` "load byte"
 - `sb x5, 0(x6)` "store byte"
- Per accedere alla half-word (16 bit) ci sono
 - (Utile per le stringhe di caratteri UNICODE, es. in Java)
 - `lh x5, 0(x6)` "load half-word"
 - `sh x5, 0(x6)` "store half-word"
- Per accedere alla word (32 bit) ci sono
 - `lw x5, 0(x6)` "load word"
 - `sw x5, 0(x6)` "store word"

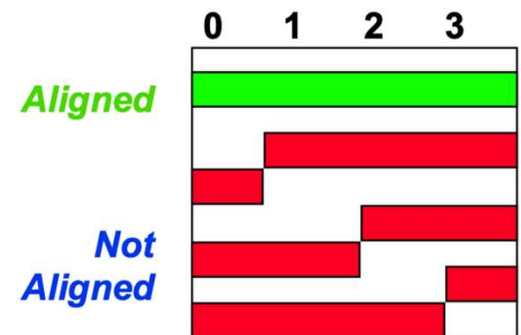
Nota: in fase di caricamento (load), dovendo porre la quantità da 8/16/32 bit in 64 bit, viene automaticamente effettuata l'**estensione del segno**. Se ciò non si vuole, si devono usare `lbu` (al posto di `lb`) e `lhu` (al posto di `lh`) e `lwu` (al posto di `lw`) ed estensione con 0

Da ora in poi...

- FAREMO RIFERIMENTO ESCLUSIVAMENTE ALLA RV32I

Restrizioni sull'allineamento degli indirizzi

- La memoria è classicamente "indirizzata al byte"
- Quindi, le istruzioni di load e store usano l'indirizzo di un byte, però
 - `lw` e `sw` trasferiscono 32 bit
 - `lh`, `lhu` e `sh` trasferiscono 16 bit
 - solo `lb`, `lbu`, `sb` trasferiscono 8 bit
- È conveniente pertanto che l'indirizzo sia opportunamente allineato...
 - per `lw` e `sw` dovrebbe essere allineato ad un multiplo di 4
 - per `lh`, `lhu`, `sh` dovrebbe essere allineato ad un multiplo di 2
- Esempi di dati ALLINEATI e NON ALLINEATI "alla word"



Nota: se si specifica un indirizzo non allineato rispetto a quanto l'istruzione desidera, il RISC-V impiegherà un tempo per l'accesso al dato maggiore

Istruzioni di accesso alla memoria

- Un esempio con variabili a 16 bit

registro base

offset

```
short g,h,f;  
short v[10];  
...  
g = h + v[3]  
v[6] = g - f
```

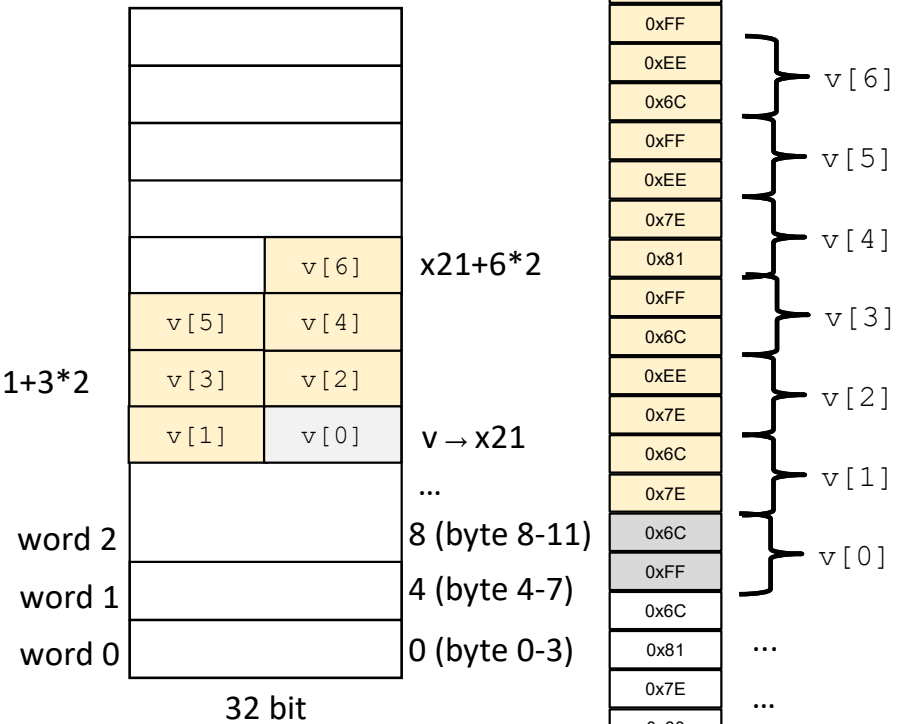
```
g → x5
h → x9
v → x21
f → x19
```

```
lh x10, 6(x21)
add x5, x9, x10
sub x6, x5, x19
sh x6, 12(x21)
```

Linguaggio C

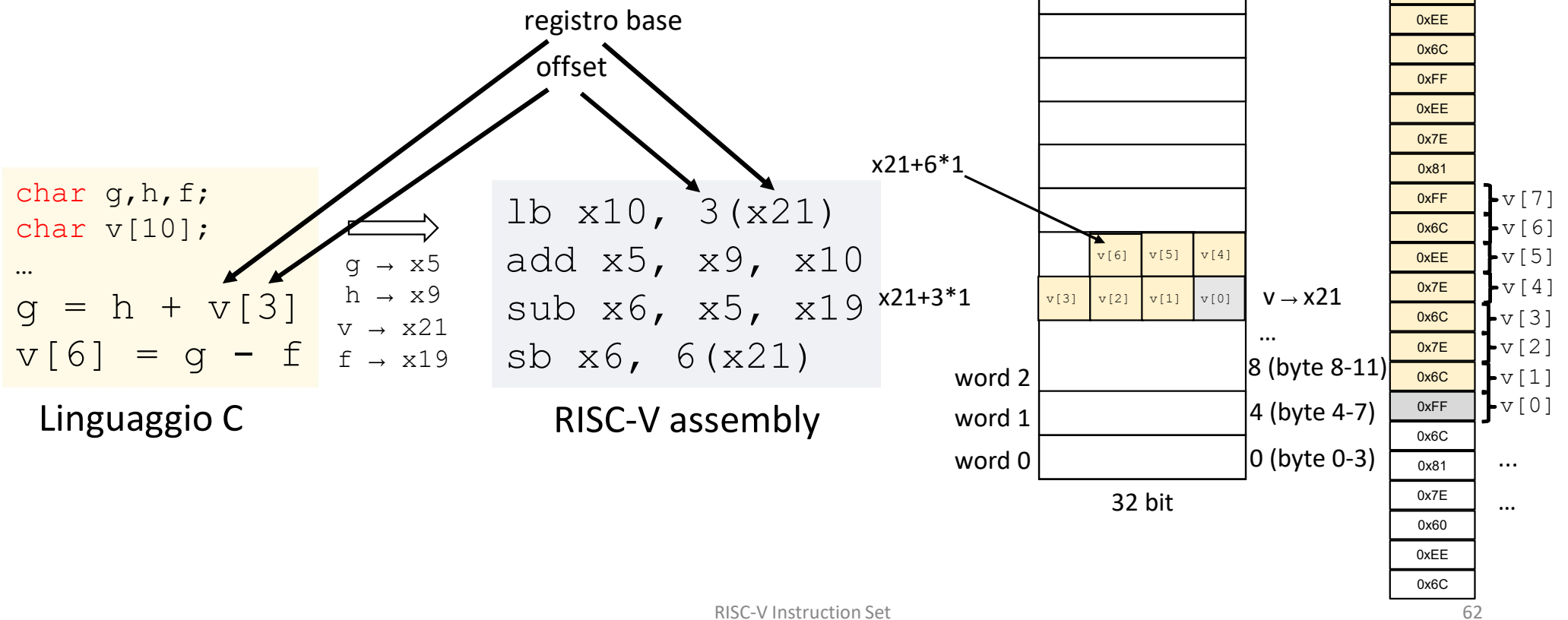
RISC-V assembly

Memoria



Istruzioni di accesso alla memoria

- Un esempio con variabili a 8 bit



Operandi immediati e costanti

- In più della metà delle operazioni aritmetiche, uno degli operandi è una costante (benchmark SPEC CPU2006)
- I valori delle costanti solitamente sono molto piccoli
 - $a = a + 1$
 - $b = b + 5$
- Es: l'operazione $b = b + 5$ potrebbe essere rappresentata con una sequenza di due istruzioni assembly

```
b = b + 5
```

Linguaggio C

$\Rightarrow$
 $b \rightarrow x5$

```
lw x9, indirizzoCostante5(x3)  
add x5, x5, x9
```

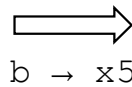
RISC-V assembly

Operandi immediati e costanti

- Alternativa: istruzioni aritmetiche in cui uno degli operandi è una costante
- L'istruzione di somma immediata è chiamata `addi` (`add immediate`)

```
b = b + 5
```

Linguaggio C



```
addi x5, x5, 5
```

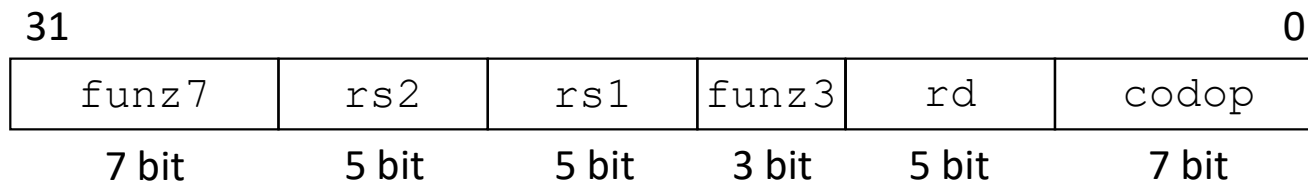
RISC-V assembly

- La costante può assumere valori tra -2048 e $+2047$
- La sottrazione immediata non esiste: si usano le costanti con valore negativo

Il linguaggio macchina

- Il linguaggio assembly fornisce una rappresentazione "human readable" (mnemonica, non numerica) delle istruzioni RISC-V
- Il calcolatore può eseguire solo istruzioni rappresentate come sequenze di bit (formato binario)
- RISC-V definisce diversi formati di istruzioni che consentono di codificare in binario ogni istruzione assembly
- Ogni istruzione RISC-V richiede esattamente 32 bit per la sua rappresentazione in linguaggio macchina (sia RV32I che RV64I)
 - La semplicità favorisce la regolarità
- Una sequenza di istruzioni in linguaggio macchina viene chiamata codice macchina

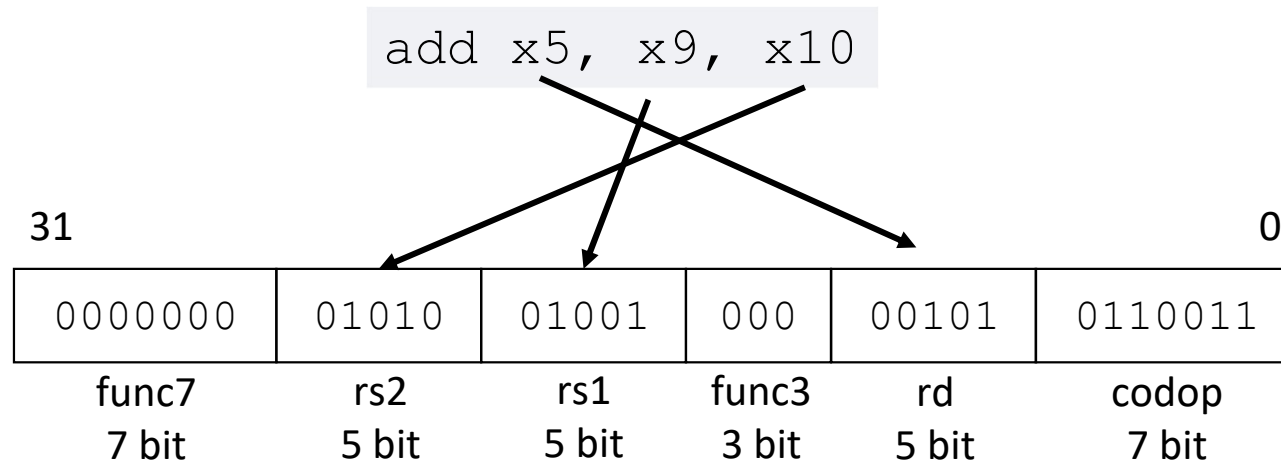
Formato di tipo R (registro)



- Permette di codificare le istruzioni `add`, `sub`, `and`, `or`, `xor`, ...
 - `codop`: codice operativo dell'istruzione
 - `rd`: registro di destinazione
 - `rs1`: registro che contiene il primo operando sorgente
 - `rs2`: registro che contiene il secondo operando sorgente
 - `funz3`, `funz7`: codici operativi aggiuntivi

Formato di tipo R (registro)

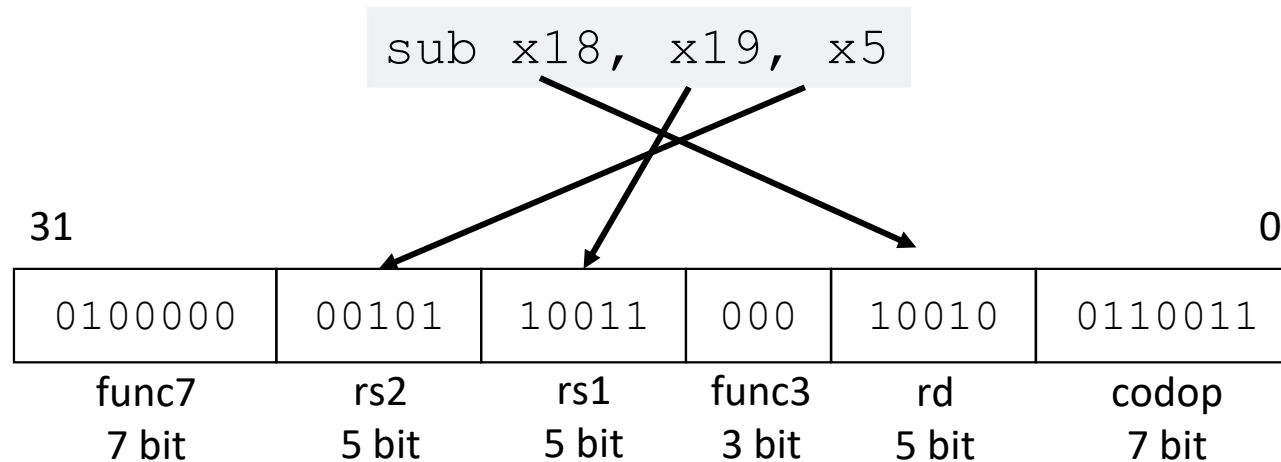
- Esempio



- Per specificare uno dei 32 registri sono necessari 5 bit
- codop + func7 + func3 indicano l'istruzione rappresentata

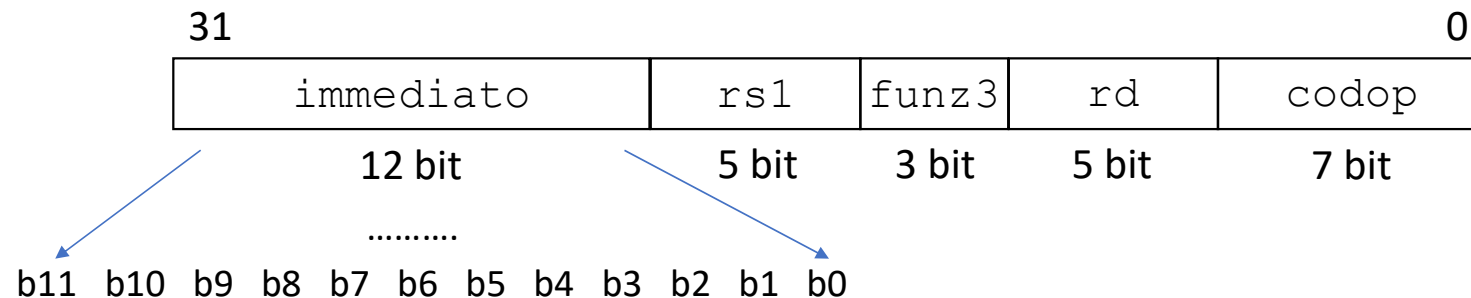
Formato di tipo R (registro)

- Esempio



- Per specificare uno dei 32 registri sono necessari 5 bit
- codop + func7 + func3 indicano l'istruzione rappresentata

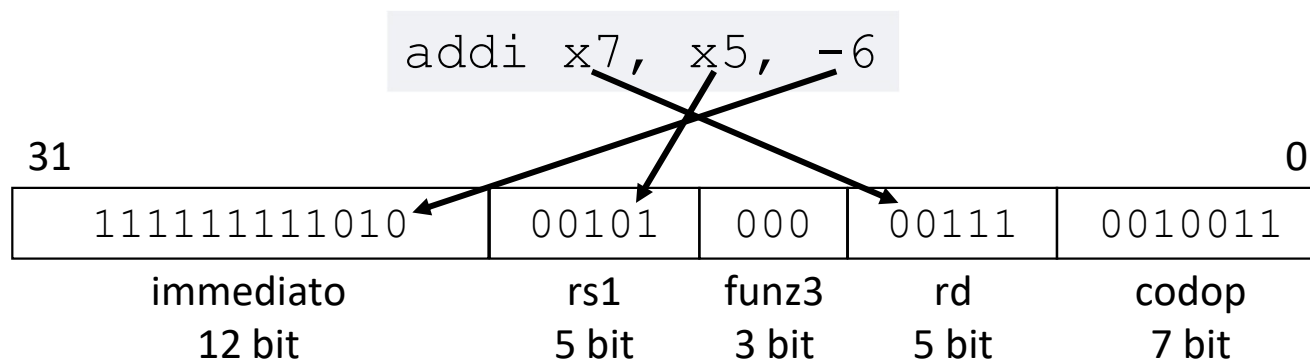
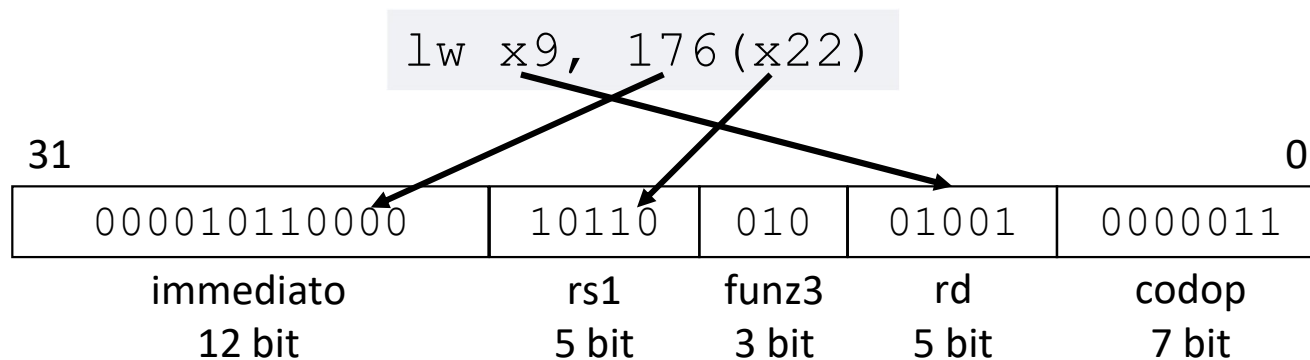
Formato di tipo I (immediato)



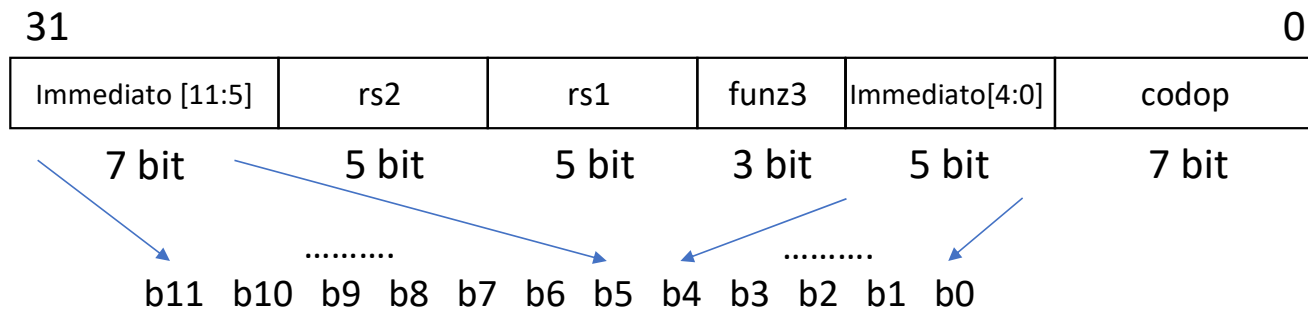
- Permette di codificare le istruzioni che richiedono il caricamento dalla memoria o una costante, come `load`, `addi`, `andi`, `ori`, ...
- Sono presenti 12 bit perché con 5 bit l'intervallo di rappresentazione per costanti e (soprattutto) offset sarebbe stato troppo ridotto
- Il campo immediato
 - Rappresentato in complemento a due
 - Valori possibili: da -2048 a $+2047$

Formato di tipo I (immediato)

- Esempi

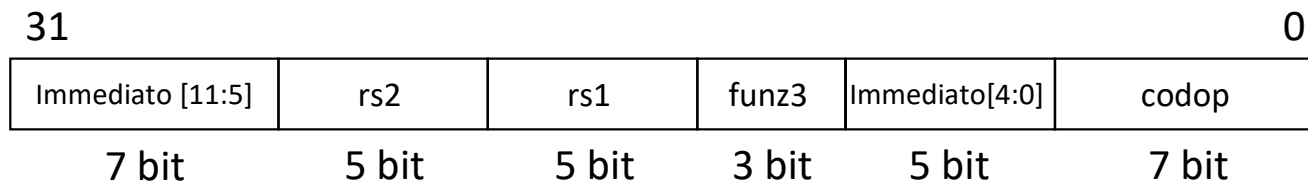


Formato di tipo S



- Permette di codificare le istruzioni che richiedono il salvataggio in memoria o una costante, come `store`
- Perché il campo immediato viene diviso in due parti?
- Il campo immediato
 - Rappresentato in complemento a due
 - Valori possibili: da -2048 a $+2047$

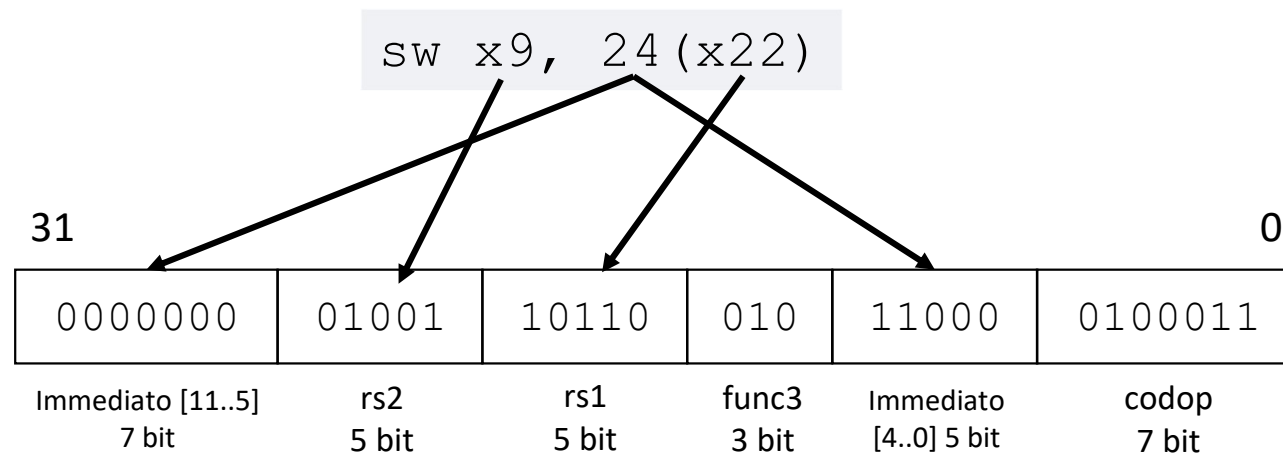
Formato di tipo S



- Permette di codificare le istruzioni che richiedono il salvataggio in memoria o una costante, come `store`
- Il campo immediato (offset) viene diviso in due parti per mantenere i due campi `rs1` e `rs2` nella stessa posizione rispetto alle istruzioni di tipo R
- Il campo immediato
 - Rappresentato in complemento a due
 - Valori possibili: da -2048 a $+2047$

Formato di tipo S

- Esempio



Manuale di riferimento RISC-V

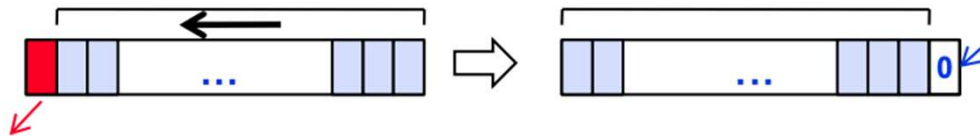
- Il libro di testo contiene tutti i dettagli sulla codifica in linguaggio macchina delle istruzioni assembly. La tabella include anche `ld` ed `sd` per la RV64I.

ISTRUZIONI ORDINATE NUMERICAMENTE PER CODICE OPERATIVO					
Mnemonico	FMT	Codice operativo	Funz3	Funz7 o cost	Esadecimale
lb	I	0000011	000		03/0
lh	I	0000011	001		03/1
lw	I	0000011	010		03/2
ld	I	0000011	011		03/3
lbu	I	0000011	100		03/4
lhu	I	0000011	101		03/5
lwu	I	0000011	110		03/6
fence	I	0001111	000		0F/0
fence.i	I	0001111	001		0F/1
addi	I	0010011	000		13/0
slli	I	0010011	001	0000000	13/1/00
slti	I	0010011	010		13/2
sltiu	I	0010011	011		13/3
xori	I	0010011	100		13/4
srli	I	0010011	101	0000000	13/5/00
srai	I	0010011	101	0100000	13/5/20
ori	I	0010011	110		13/6
andi	I	0010011	111		13/7
auipc	U	0010111			17
addiw	I	0011011	000		1B/0
slliw	I	0011011	001	0000000	1B/1/00
srliw	I	0011011	101	0000000	1B/5/00
sraiw	I	0011011	101	0100000	1B/5/20
sb	S	0100011	000		23/0
sh	S	0100011	001		23/1
sw	S	0100011	010		23/2
sd	S	0100011	011		23/3
add	R	0110011	000	0000000	33/0/00
sub	R	0110011	000	0100000	33/0/20
sll	R	0110011	001	0000000	33/1/00
slt	R	0110011	010	0000000	33/2/00

Operazioni logiche

- Shift logico

- A sinistra



Shift Left Logical

```
sll x9, x22, x19
```

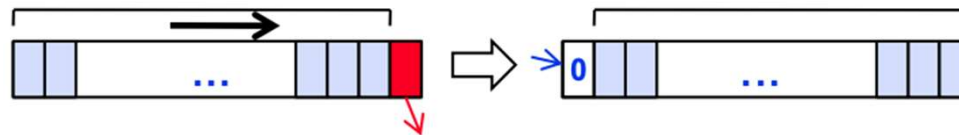
$x9 = x22 \ll x19$

```
slli x9, x22, 5
```

$x9 = x22 \ll 5$

Shift Left Logical Immediate

- A destra



Shift Right Logical

```
srl x9, x22, x19
```

$x9 = x22 \gg x19$

```
srli x9, x22, 5
```

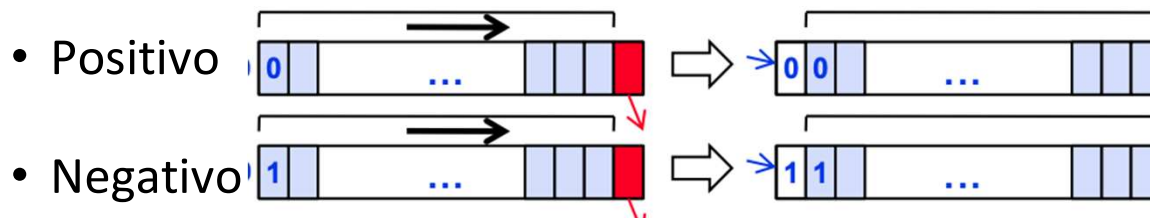
$x9 = x22 \gg 5$

Shift Right Logical Immediate

Operazioni logiche

- Shift aritmetico

- A destra



- A sinistra

- Non esiste perché non ha senso: identico a `sll`

Shift Right Arithmetic

```
sra x9, x22, x19  
x9 = x22 >> x19
```

```
srai x9, x22, 5  
x9 = x22 >> 5
```

Shift Right Arithmetic Immediate

Operazioni logiche

- Le istruzioni assembly `sll` e `srl` e `sra` si rappresentano in linguaggio macchina con il formato R
- Le istruzioni assembly `slli` e `srl` e `srai` si rappresentano in linguaggio macchina con il formato I (vengono utilizzati i bit meno significativi del campo immediato per codificare la costante, gli altri sono posti a zero)
- Lo shift a sinistra di k posizioni calcola una moltiplicazione per 2^k
- Lo shift a destra aritmetico di k posizioni calcola una divisione intera per 2^k nel caso di numeri senza segno
- Lo shift a destra aritmetico di k posizioni non sempre calcola una divisione intera per 2^k nel caso di numeri in complemento a 2. Nel caso di numeri negativi si devono compiere operazioni preliminari (vedi laboratorio)

Prima un esempio semplice...

```
int i,j;  
int v[10];  
  
...  
v[i]=v[j];
```

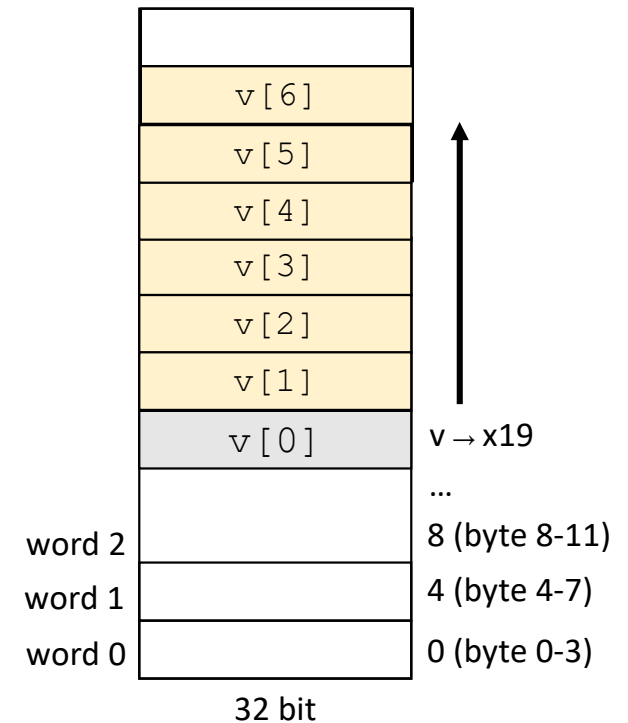
Linguaggio C

→
i → x9
j → x21
v → x19

?

RISC-V assembly

Memoria



Prima un esempio semplice...

x6 contiene l'indirizzo del primo byte che rappresenta v[j]

```
int i,j;  
int v[10];
```

```
...  
v[i]=v[j];
```

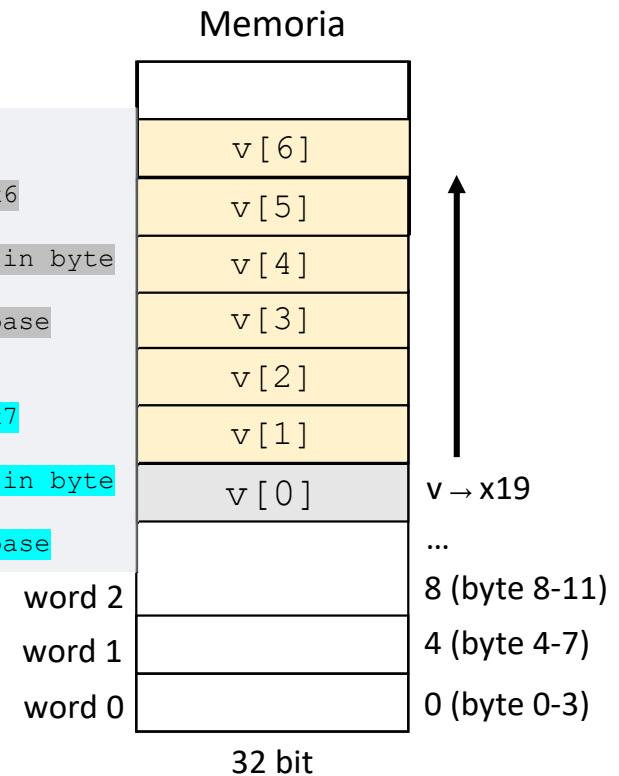
Linguaggio C

⇒
i → x9
j → x21
v → x19

```
...  
addi x6,x21,0  
slli x6,x6,2  
add x6,x6,x19  
lw x6,0(x6)  
addi x7,x9,0  
slli x7,x7,2  
add x7,x7,x19  
sw x6,0(x7)
```

RISC-V assembly

Calcola indice in x6
Moltiplica per dim in byte
Somma l'indirizzo base
Calcola indice in x7
Moltiplica per dim in byte
Somma l'indirizzo base

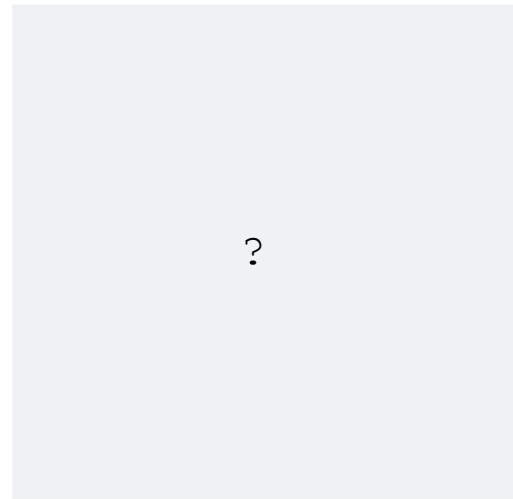
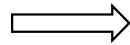


x7 contiene l'indirizzo del primo byte che rappresenta v[i]

Poi un esempio più complesso...

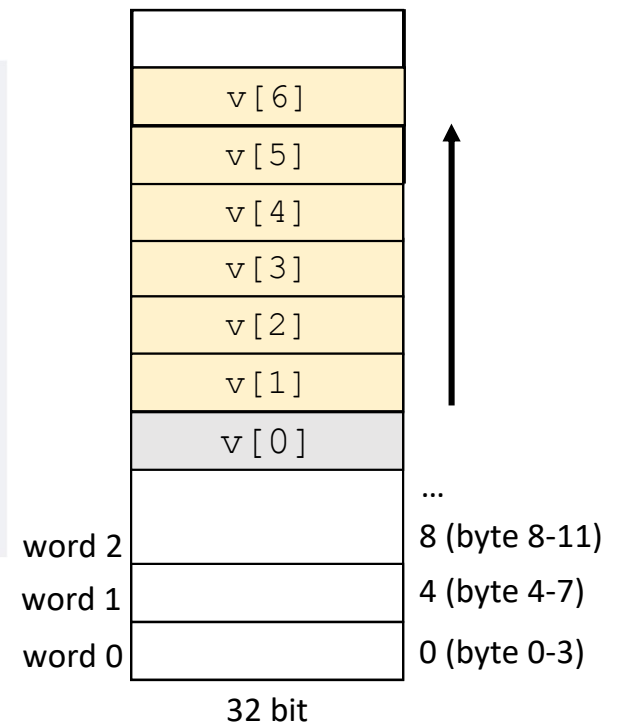
```
int d,i,j;  
int v[10];  
...  
j=5;  
...  
v[i+d]=v[j+2];
```

Linguaggio C



RISC-V assembly

Memoria



x6 contiene l'indirizzo del primo byte che rappresenta v[j+2]

Poi un esempio più complesso...

```
int d,i,j;  
int v[10];  
...  
j=5;  
...  
v[i+d]=v[j+2];
```

Linguaggio C

→
d → x5
i → x9
j → x21
v → x19

```
addi x21,x0,5
```

...

```
addi x6,x21,2
```

```
slli x6,x6,2
```

```
add x6,x6,x19
```

```
lw x6,0(x6)
```

```
add x7,x9,x5
```

```
slli x7,x7,2
```

```
add x7,x7,x19
```

```
sw x6,0(x7)
```

RISC-V assembly

Calcola espressione per j

...

Calcola indice in x6

Moltiplica per dim in byte

Somma l'indirizzo base

Calcola indice in x7

Moltiplica per dim in byte

Somma l'indirizzo base

word 2

word 1

word 0

Memoria

v[6]

v[5]

v[4]

v[3]

v[2]

v[1]

v[0]

v → x19

...
8 (byte 8-11)

4 (byte 4-7)

0 (byte 0-3)

32 bit

x7 contiene l'indirizzo del primo byte che rappresenta v[i+d]

Operazioni logiche

- AND

```
and x9,x22,x19  
x9 = x22 & x19
```

```
andi x9,x22,5  
x9 = x22 & 5
```

- OR

```
or x9,x22,x19  
x9 = x22 | x19
```

```
ori x9,x22,5  
x9 = x22 | 5
```

- XOR

```
xor x9,x22,x19  
x9 = x22  $\oplus$  x19
```

```
xori x9,x22,5  
x9 = x22  $\oplus$  5
```

- NOT

Pseudoistruzione

```
not x5,x6  
x5 =  $\overline{x6}$ 
```

Operazioni logiche

- Le istruzioni assembly `and` e `or` e `xor` si rappresentano in linguaggio macchina con il formato R
- Le istruzioni assembly `andi` e `ori` e `xori` si rappresentano in linguaggio macchina con il formato I

Operazioni logiche

- L'istruzione `and` permette di selezionare alcuni bit del primo operando indicandoli all'interno di una maschera (secondo operando)
- Esempio

```
and x5, x6, x7
```

x6	00100100 00010101 00001011 10100110	Sorgente
x7	00000100 00000110 00000010 00010010	Maschera
x5	00000100 00000100 00000010 00000010	Risultato

Operazioni logiche

- L'istruzione `or` permette di ricopiare il primo operando, settando ad uno anche i bit che sono specificati nella maschera indicata come secondo operando
- Esempio

```
or x5, x6, x7
```

x6	00100100 00010101 00001011 10100110	Sorgente
x7	00000100 00000110 01000010 00010010	Maschera
x5	00100100 00010111 01001011 10110110	Risultato

Salti condizionati

- Permettono di variare il flusso del programma (variando il valore del PC) al verificarsi di una condizione

```
beq rs1,rs2,L1
```

Branch if EQual

- Il flusso di programma continua all'istruzione con etichetta `L1` se il valore del registro `rs1` è uguale a quello di `rs2`

```
bne rs1,rs2,L1
```

Branch if Not Equal

- Il flusso di programma continua all'istruzione con etichetta `L1` se il valore del registro `rs1` è diverso a quello di `rs2`

Salti condizionati

- Permettono di variare il flusso del programma (variando il valore del PC) al verificarsi di una condizione

```
blt rs1,rs2,L1
```

Branch if Less Than

- Il flusso di programma continua all'istruzione con etichetta `L1` se il valore del registro `rs1` è minore a quello di `rs2`

```
bge rs1,rs2,L1
```

Branch if Greater than
or Equal

- Il flusso di programma continua all'istruzione con etichetta `L1` se il valore del registro `rs1` è maggiore o uguale a quello di `rs2`

Salti condizionati

- Esistono anche le operazioni di salto condizionato che confrontano i due registri `rs1` e `rs2` trattandoli come numeri senza segno

```
bltu rs1,rs2,L1
```

Branch if Less Than Unsigned

```
bgeu rs1,rs2,L1
```

Branch if Greater than or Equal Unsigned

Salti condizionati: costrutto if-then-else

- Attraverso le istruzioni `beq` e `bne` è possibile tradurre in assembly il costrutto `if` dei linguaggi di programmazione ad alto livello

La scelta di test per not equal è più conveniente in questo caso

```
if (i==j)
    f=g+h;
else
    f=g-h;
```

Linguaggio C

→
f → x19
g → x20
h → x21
i → x22
j → x23

```
bne x22,x23,ELSE
add x19,x20,x21
beq x0,x0,ENDIF
ELSE: sub x19,x20,x21
ENDIF:
```

RISC-V assembly

Salto incondizionato

Salti condizionati: ciclo for

- Una possibile implementazione

```
for (i=0; i<100; i++)  
{  
    ...  
}
```

$i \rightarrow x19$
→

?

Salti condizionati: ciclo for

- Una possibile implementazione

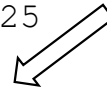
```
for (i=0;i<100;i++)  
{  
    ...  
}
```

$i \rightarrow x19$
→

```
FOR:      add x19,x0,x0  
          addi x20,x0,100  
          bge x19,x20,ENDFOR  
          ...  
          addi x19,x19,1  
          beq x0,x0,FOR  
ENDFOR:
```

Salti condizionati: ciclo while

i → x22
k → x24
v → x25



```
int v[10], k, i;  
while (v[i]==k)  
{  
    ...  
    i=i+1;  
}
```

?

Salti condizionati: ciclo while

```
int v[10], k, i;  
while (v[i]==k)  
{  
    ...  
    i=i+1;  
}
```

i → x22
k → x24
v → x25



```
LOOP:    slli x10,x22,2  ← Salva in x10 l'indirizzo della word v[i]  
         add x10,x10,x25  
  
         lw x9,0(x10)    ← Carica dalla memoria il valore contenuto nella  
                           word v[i]  
  
         bne x9,x24,ENDLOOP ← Se v[i] != k, allora il ciclo termina  
         ...  
         addi x22,x22,1  ← Incrementa i e torna alla valutazione della  
         beq x0,x0,LOOP   condizione del ciclo while  
ENDLOOP:
```

Salti condizionati

- L'istruzione `slt` (esiste anche `slti`) scrive 1 in `rd` se `rs1 < rs2` e scrive 0 altrimenti
- Permette di costruire strutture di controllo con istruzioni di salto generiche

```
if (i<j)
    k=1;
else
    k=0;
```

⇒
i → x19
j → x20
k → x21

```
slt x21, x19, x20
```

← Tipo R in linguaggio
macchina

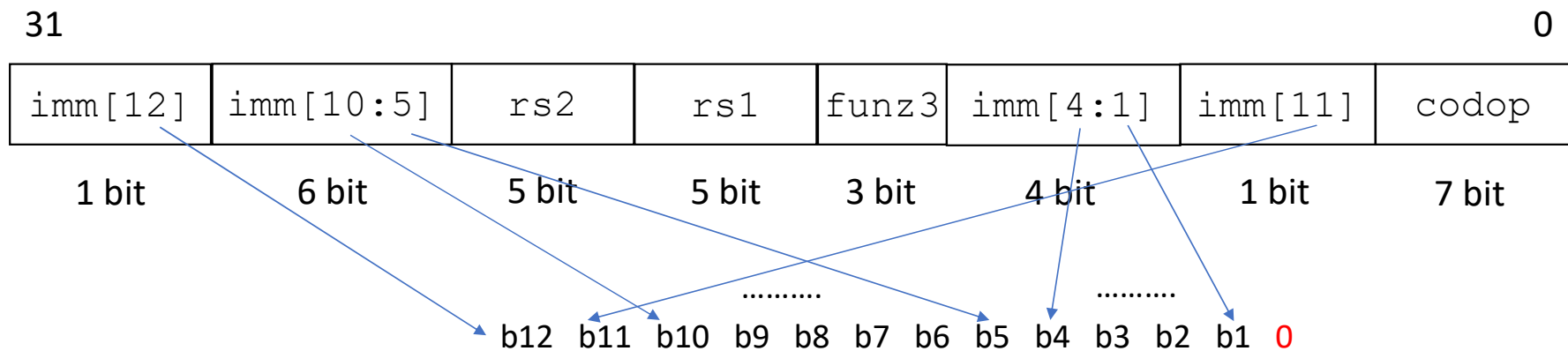
- Possiamo, ad esempio, inserire dopo `slt` l'istruzione `beq rs1, x0, L1`
 - per il confronto su "`>=`" basta invertire la condizione (`bne`)
 - per il confronto su "`>`" basta scambiare gli operandi della `slt`
 - per il confronto su "`<=`", inverti condizione e scambio operandi

Salti condizionati

- Le istruzioni aritmetiche fin qui esaminate operano su interi con segno
 - Gli operandi (a 32 bit, nei registri) sono rappresentati in complemento a due: i valori vanno da -2^{31} a $2^{31}-1$
 - Anche i byte-offset di `lw` e `sw` ed il numero di istruzioni di cui saltare nella `bne/beq` sono interi con segno da -2^{11} a $2^{11}-1$
- È possibile utilizzare anche operandi senza segno
 - In tal caso i valori vanno da 0 a $2^{32}-1$
- Invece di `slt`, `slti` si useranno `sltu`, `sltiu`
 - `slt` e `sltu` forniranno un risultato diverso se un operando è negativo (es. $-4 < 3$ a 1 con `slt`, ma 0 con `sltu`)
 - Infatti il -4 verrebbe interpretato come un numero molto grande, quindi $7 < 3$ risulterebbe falso

Salti condizionati e linguaggio macchina

- Le istruzioni di salto condizionato utilizzano il **formato di tipo SB**
- Il formato può rappresentare indirizzi di salto da -4096 a 4094, in multipli di due
- Infatti, abbiamo 13 bit in complemento a 2 con solo i numeri pari



Salti condizionati e linguaggio macchina

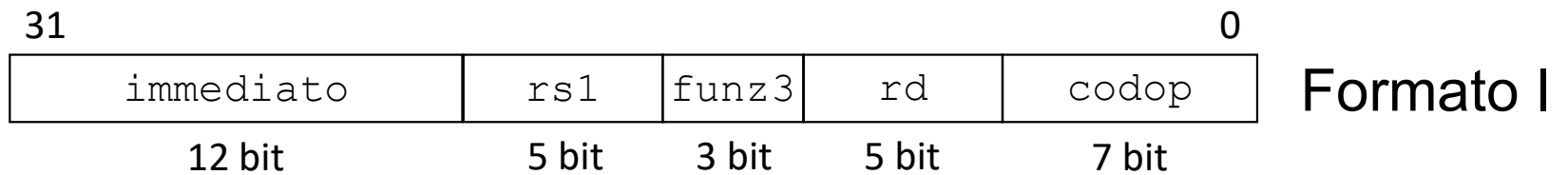
- Si utilizza l'indirizzamento relativo al program counter (PC-relative addressing)
- Il campo immediato di 12+1 bit contiene l'offset rispetto al valore di PC (espresso in complemento a due)
- Per ottenere il prossimo valore di PC in caso di salto, viene eseguito il calcolo

$$PC = PC + \text{immediato}$$

Salti condizionati e linguaggio macchina

- Le istruzioni macchina RISC-V hanno una dimensione di 32 bit
- I possibili valori di scostamento sono compresi tra -4096 e + 4094 ($[-2048*2, +2047*2]$)
- Non c'è un vincolo di allineamento a 32 bit, ma questo sistema consente solo di saltare ad indirizzi di memoria pari (perché b0 è implicitamente uguale a 0)
- Perché il moltiplicatore non è 4? I progettisti hanno voluto prevedere la possibilità di rappresentare le istruzioni macchina anche su 16 bit
- Noi, però, continueremo sempre a considerare istruzioni su 32 bit. Il compilatore genererà campi immediati di istruzioni di salto sempre multipli di 4, ovvero con b1 = 0 e con b0 implicitamente a 0.

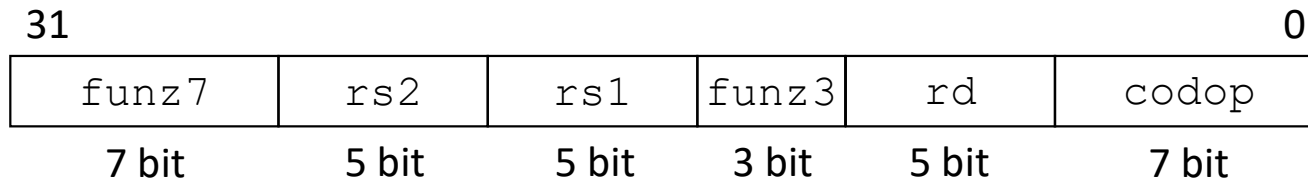
Salti condizionati e linguaggio macchina



Indirizzo		Istruzione					
80000		0000000	00010	10110	001	01010	0010011
80004		0000000	11001	01010	000	01010	0110011
80008		0000000	00000	01010	011	01001	0000011
80012		0000000	11000	01001	001	01100	1100011
80016		0000000	00001	10110	000	10110	0010011
80020		1111111	00000	00000	000	01101	1100011

LOOP:	slli x10,x22,2	
	add x10,x10,x25	
	lw x9,0(x10)	
	bne x9,x24,ENDLOOP	
	addi x22,x22,1	
	beq x0,x0,LOOP	
ENDLOOP:	PC = 80000	

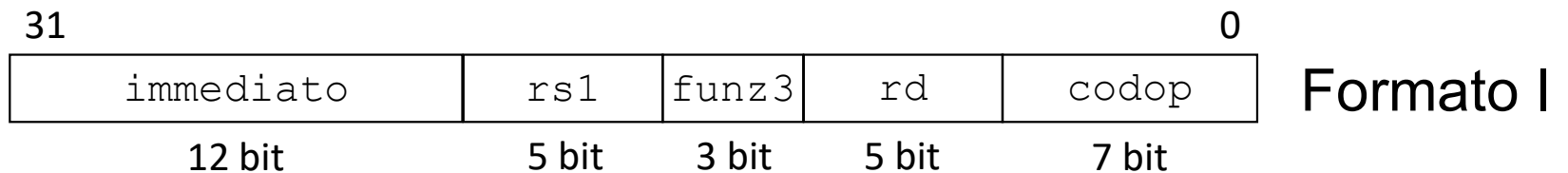
Salti condizionati e linguaggio macchina



Formato R

		Indirizzo	Istruzione				
LOOP:	slli x10,x22,2	80000	0000000	00010	10110	001	01010 0010011
	add x10,x10,x25	80004	0000000	11001	01010	000	01010 0110011
	lw x9,0(x10)	80008	0000000	00000	01010	011	01001 0000011
	bne x9,x24,ENDLOOP	80012	0000000	11000	01001	001	01100 1100011
	addi x22,x22,1	80016	0000000	00001	10110	000	10110 0010011
	beq x0,x0,LOOP	80020	1111111	00000	00000	000	01101 1100011
ENDLOOP:	PC = 80004						

Salti condizionati e linguaggio macchina



		Indirizzo	Istruzione					
LOOP:	slli x10,x22,2	80000	0000000	00010	10110	001	01010	0010011
	add x10,x10,x25	80004	0000000	11001	01010	000	01010	0110011
	lw x9,0(x10)	80008	0000000	00000	01010	011	01001	0000011
	bne x9,x24,ENDLOOP	80012	0000000	11000	01001	001	01100	1100011
	addi x22,x22,1	80016	0000000	00001	10110	000	10110	0010011
	beq x0,x0,LOOP	80020	1111111	00000	00000	000	01101	1100011
ENDLOOP:	PC = 80008							

Salti condizionati e linguaggio macchina

31

0

imm[12]	imm[10:5]	rs2	rs1	funz3	imm[4:1]	imm[11]	codop
1 bit	6 bit	5 bit	5 bit	3 bit	4 bit	1 bit	7 bit

		Indirizzo	Istruzione				
LOOP:	slli x10,x22,2	80000	0000000	00010	10110	001	01010 0010011
	add x10,x10,x25	80004	0000000	11001	01010	000	01010 0110011
	lw x9,0(x10)	80008	0000000	00000	01010	011	01001 0000011
	bne x9,x24,ENDLOOP	80012	0000000	11000	01001	001	01100 1100011
	addi x22,x22,1	80016	0000000	00001	10110	000	10110 0010011
	beq x0,x0,LOOP	80020	1111111	00000	00000	000	01101 1100011
ENDLOOP:	PC = 80012						

Salti condizionati e linguaggio macchina

31

0

imm[12]	imm[10:5]	rs2	rs1	funz3	imm[4:1]	imm[11]	codop
---------	-----------	-----	-----	-------	----------	---------	-------

1 bit

6 bit

5 bit

5 bit

3 bit

4 bit

1 bit

7 bit

Indirizzo	Istruzione						
80000	0000000	00010	10110	001	01010	0010011	
80004	0000000	11001	01010	000	01010	0110011	
80008	0000000	00000	01010	011	01001	0000011	
80012	0000000	11000	01001	001	01100	1100011	
80016	0000000	00001	10110	000	10110	0010011	
80020	1111111	00000	00000	000	01101	1100011	

LOOP: slli x10,x22,2
 add x10,x10,x25
 lw x9,0(x10)
 bne x9,x24,ENDLOOP
 addi x22,x22,1
 beq x0,x0,LOOP

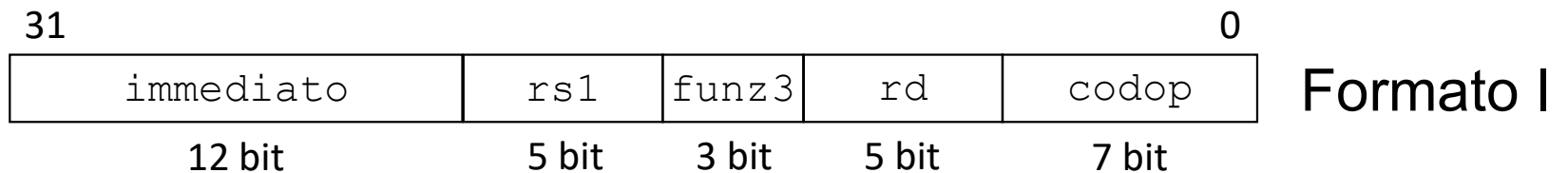
 ENDLOOP: **PC = 80012 + 12 = 80024**

Se la condizione di bne è vera

b12 b11 b10 b9 b8 b7 b6 b5 b4 b3 b2 b1 b0
 0 0 0 0 0 0 0 0 0 1 1 0 0 = 12

RISC-V Instruction Set

Salti condizionati e linguaggio macchina



		Indirizzo	Istruzione				
LOOP:	slli x10,x22,2	80000	0000000	00010	10110	001	01010 0010011
	add x10,x10,x25	80004	0000000	11001	01010	000	01010 0110011
	lw x9,0(x10)	80008	0000000	00000	01010	011	01001 0000011
	bne x9,x24,ENDLOOP	80012	0000000	11000	01001	001	01100 1100011
	addi x22,x22,1	80016	0000000	00001	10110	000	10110 0010011
	beq x0,x0,LOOP	80020	1111111	00000	00000	000	01101 1100011
ENDLOOP:	PC = 80016						

Se la condizione di bne è falsa

Salti condizionati e linguaggio macchina

31

0

imm[12]	imm[10:5]	rs2	rs1	funz3	imm[4:1]	imm[11]	codop
1 bit	6 bit	5 bit	5 bit	3 bit	4 bit	1 bit	7 bit

		Indirizzo	Istruzione				
LOOP:	slli x10,x22,2	80000	0000000	00010	10110	001	01010 0010011
	add x10,x10,x25	80004	0000000	11001	01010	000	01010 0110011
	lw x9,0(x10)	80008	0000000	00000	01010	011	01001 0000011
	bne x9,x24,ENDLOOP	80012	0000000	11000	01001	001	01100 1100011
	addi x22,x22,1	80016	0000000	00001	10110	000	10110 0010011
	beq x0,x0,LOOP	80020	1111111	00000	00000	000	01101 1100011
ENDLOOP:	PC = 80020						

Se la condizione di bne è falsa

Salti condizionati e linguaggio macchina

31

0

imm[12]	imm[10:5]	rs2	rs1	funz3	imm[4:1]	imm[11]	codop
---------	-----------	-----	-----	-------	----------	---------	-------

1 bit

6 bit

5 bit

5 bit

3 bit

4 bit

1 bit

7 bit

		Indirizzo	Istruzione				
LOOP:	slli x10,x22,2	80000	0000000	00010	10110	001	01010 0010011
	add x10,x10,x25	80004	0000000	11001	01010	000	01010 0110011
	lw x9,0(x10)	80008	0000000	00000	01010	011	01001 0000011
	bne x9,x24,ENDLOOP	80012	0000000	11000	01001	001	01100 1100011
	addi x22,x22,1	80016	0000000	00001	10110	000	10110 0010011
	beq x0,x0,LOOP	80020	1111111	00000	00000	000	01101 1100011
ENDLOOP:	PC = 80020 - 20 = 80000		1 1 1 1 1 1 1 1 0 1 1 0 0 = -20				

Se la condizione di beq è vera (in questo caso lo è sempre)

RISC-V Instruction Set

106

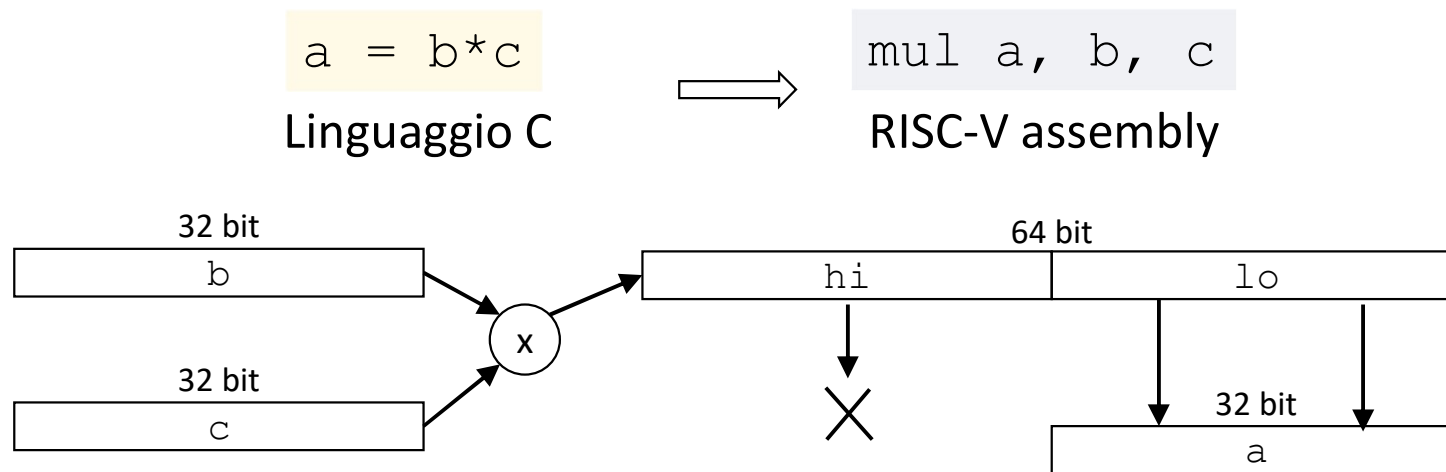
Pseudoistruzioni

- Il linguaggio assembly fornisce anche “pseudoistruzioni”
 - Una pseudoistruzione esiste solo in linguaggio assembly e non in linguaggio macchina
 - l'assemblatore traduce le pseudoistruzioni nel linguaggio macchina delle corrispondenti istruzioni
 - Per esempio, quando si effettua il debugging è necessario ricordare quali siano le istruzioni reali
 - L'elenco delle pseudoistruzioni è presente nel manuale di riferimento
- Esempio

`mv x5, x6` → `addi x5, x6, 0`

`not x5, x6` → `xori x5, x6, -1`

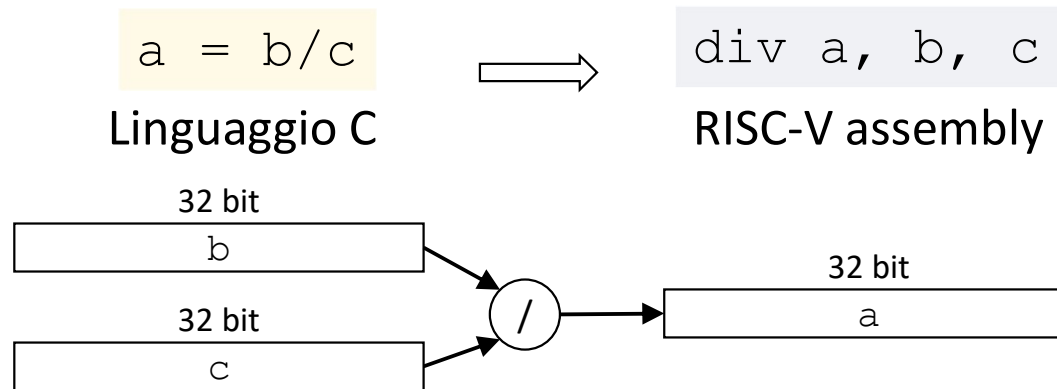
Istruzioni aritmetiche: moltiplicazione



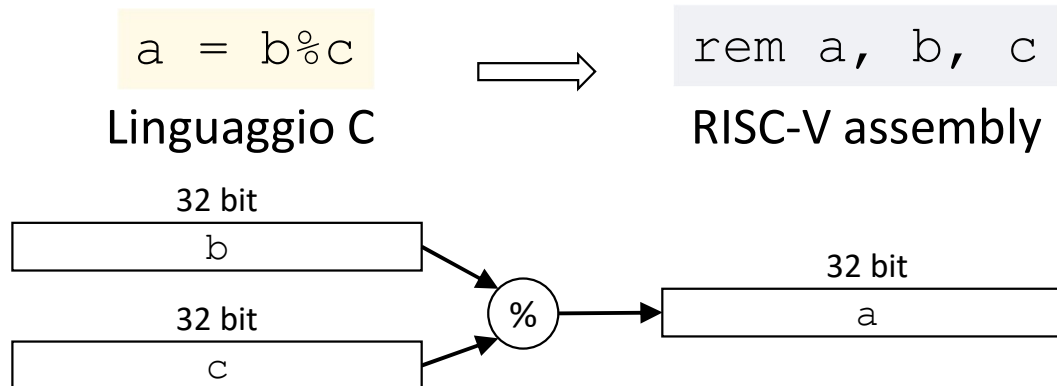
- Viene utilizzato un registro interno nascosto all'utente
- I 32 bit più significativi possono essere ottenuti con l'istruzione
- Ci sono le versioni unsigned

`mulh a, b, c`

Istruzioni aritmetiche: divisione e resto



- Ci sono le versioni unsigned



RISC-V Instruction Set